

Powershell (II)

2r.SMX - SOX Pràctica 0.8

Asignatura: Sistemes operatius en red

Profesor: Teresa Sánchez Cazorla

Alumne: Santiago García Santiago

Fecha de entrega: 15-11-2022

Índice

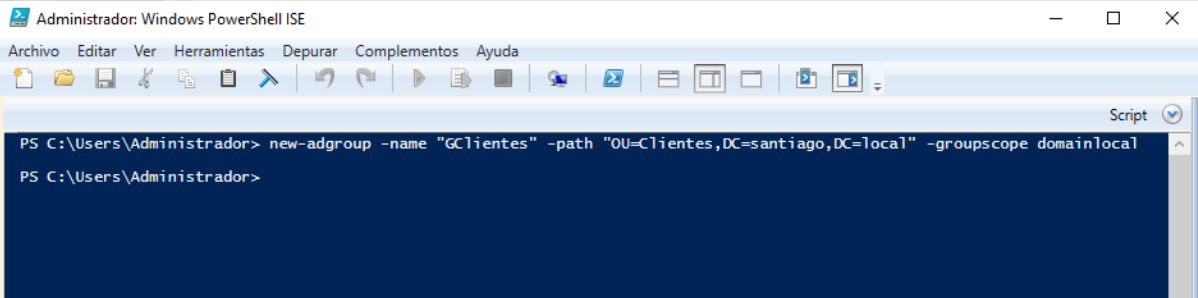
- 01- Creación de un grupo en una UO ya existente
- 02- Ver información de un grupo
- 03- Mostrar la información de un grupo concreto mediante filtro
- 04- Modificar la información de un grupo
- 05- Ver miembros de un grupo
- 06- Añadir miembros a un grupo
- 07- Añadir ordenadores a un grupo
- 08- Eliminar miembros de un grupo
- 09- Borrar un grupo
- 10- Información proporcionada por Get-ADUser
- 11- Mostrar toda la información de un usuario específico
- 12- Verificación de la versión Powershell
- 13- Listar usuarios de una UO concreta
- 14- Listar cuentas de usuario expiradas
- 15- Listar cuentas de usuario deshabilitadas
- 16- Listar Último acceso de las cuentas
- 17- Listado de usuario bloqueadas
- 18- Listado de usuarios inactivos
- 19- Listado de usuarios con password que nunca expira
- 20- Listado de usuarios con contraseñas débiles
- 21- Listado de nuevos usuarios

Índice

22- Listado de equipos inactivos del dominio

23- Listado de equipos deshabilitados

01-Creación de un grupo en una UO ya existente



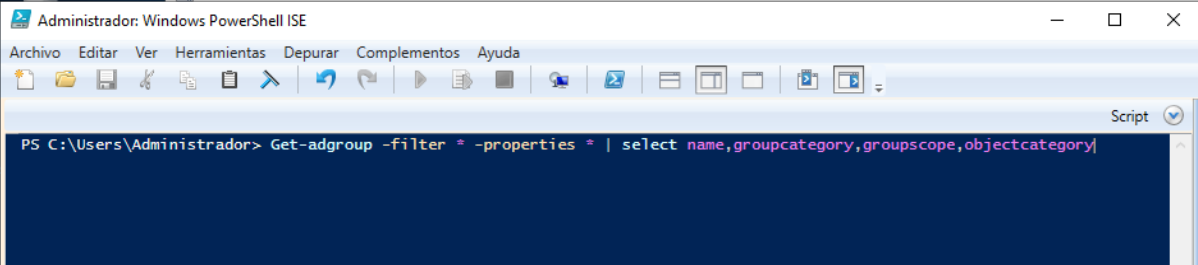
```
Administrador: Windows PowerShell ISE
Archivo Editar Ver Herramientas Depurar Complementos Ayuda
Script
PS C:\Users\Administrador> new-adgroup -name "GClientes" -path "OU=Clientes,DC=santiago,DC=local" -groupscope domainlocal
PS C:\Users\Administrador>
```

En este caso en particular, estamos creando el grupo "GCliente" en la unidad organizativa "Cliente".

```
new-adgroup -name "GClientes" -path "OU=Clientes,DC=sanchez,DC=local"
-groupscope domainlocal
```

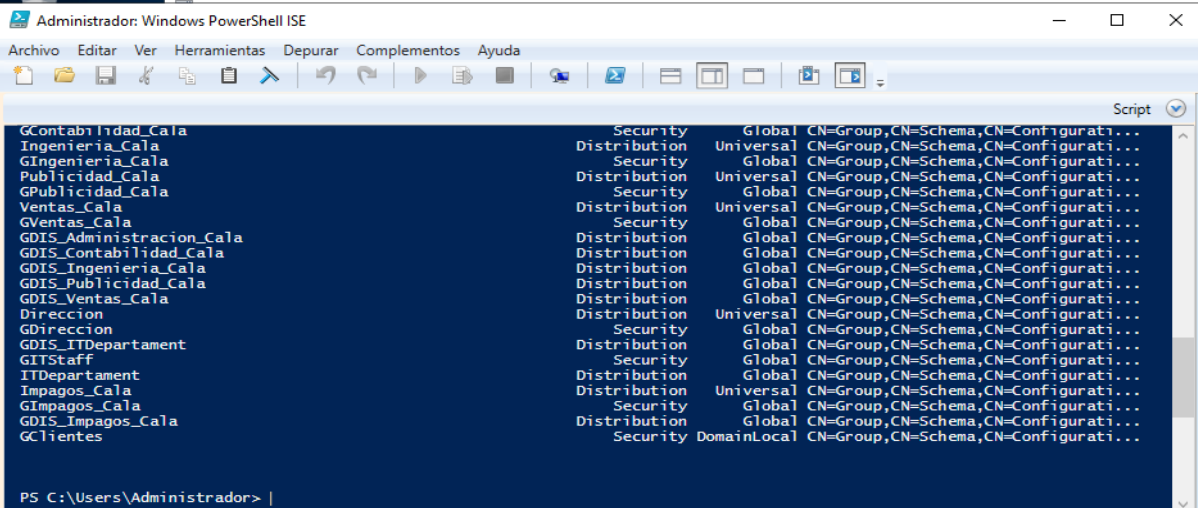
02-Ver información de un grupo

Mediante el siguiente comando podemos visualizar la información de un grupo concreto.



```
Administrador: Windows PowerShell ISE
Archivo Editar Ver Herramientas Depurar Complementos Ayuda
Script
PS C:\Users\Administrador> Get-adgroup -filter * -properties * | select name,groupcategory,groupscope,objectcategory
```

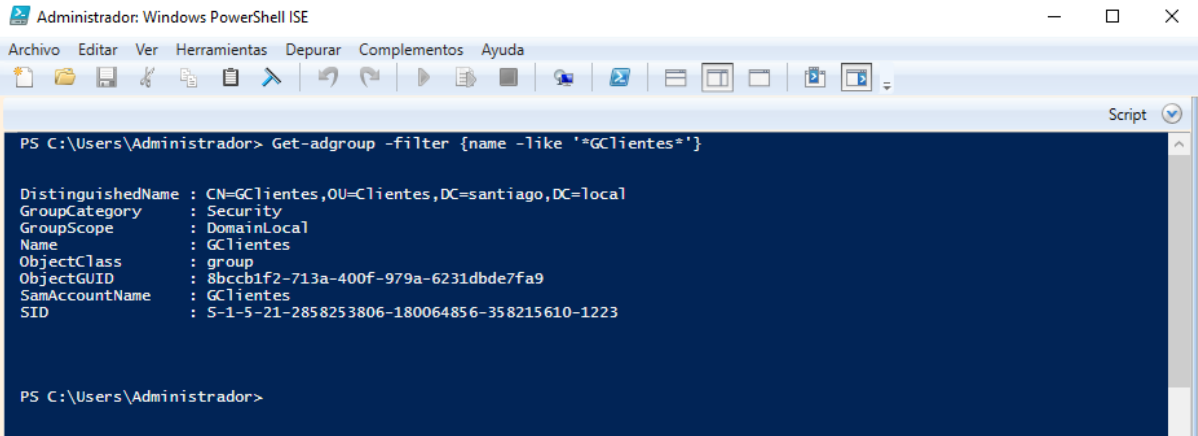
```
Get-adgroup -filter * -properties * | select
name,groupcategory,groupscope,objectcategory
```



```
Administrador: Windows PowerShell ISE
Archivo Editar Ver Herramientas Depurar Complementos Ayuda
Script
GContabilidad_Cala Security Global CN=Group,CN=Schema,CN=Configurati...
Ingenieria_Cala Distribution Universal CN=Group,CN=Schema,CN=Configurati...
Ingenieria_Cala Security Global CN=Group,CN=Schema,CN=Configurati...
Publicidad_Cala Distribution Universal CN=Group,CN=Schema,CN=Configurati...
GPublicidad_Cala Security Global CN=Group,CN=Schema,CN=Configurati...
Ventas_Cala Distribution Universal CN=Group,CN=Schema,CN=Configurati...
GVentas_Cala Security Global CN=Group,CN=Schema,CN=Configurati...
GDIS_Administracion_Cala Distribution Global CN=Group,CN=Schema,CN=Configurati...
GDIS_Contabilidad_Cala Distribution Global CN=Group,CN=Schema,CN=Configurati...
GDIS_Ingenieria_Cala Distribution Global CN=Group,CN=Schema,CN=Configurati...
GDIS_Publicidad_Cala Distribution Global CN=Group,CN=Schema,CN=Configurati...
GDIS_Ventas_Cala Distribution Global CN=Group,CN=Schema,CN=Configurati...
Direccion Distribution Universal CN=Group,CN=Schema,CN=Configurati...
GDireccion Security Global CN=Group,CN=Schema,CN=Configurati...
GDIS_ITDepartament Distribution Global CN=Group,CN=Schema,CN=Configurati...
GITStaff Security Global CN=Group,CN=Schema,CN=Configurati...
ITDepartament Distribution Global CN=Group,CN=Schema,CN=Configurati...
Impagos_Cala Distribution Universal CN=Group,CN=Schema,CN=Configurati...
GImpagos_Cala Security Global CN=Group,CN=Schema,CN=Configurati...
GDIS_Impagos_Cala Distribution Global CN=Group,CN=Schema,CN=Configurati...
GClientes Security DomainLocal CN=Group,CN=Schema,CN=Configurati...
PS C:\Users\Administrador> |
```

03- Mostrar la información de un grupo concreto mediante filtro

Mediante el siguiente comando podemos realizar una búsqueda filtrada, para localizar un grupo concreto.



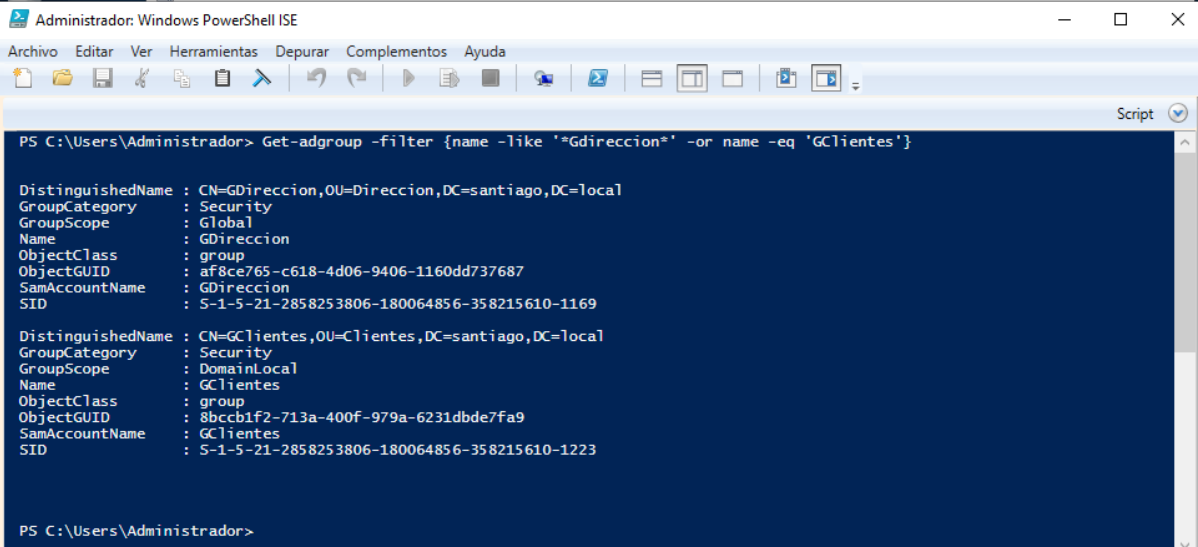
```
Administrador: Windows PowerShell ISE
Archivo Editar Ver Herramientas Depurar Complementos Ayuda
Script
PS C:\Users\Administrador> Get-adgroup -filter {name -like '*GClientes*'}

DistinguishedName : CN=GClientes,OU=Clientes,DC=santiago,DC=local
GroupCategory      : Security
GroupScope         : DomainLocal
Name               : GClientes
ObjectClass        : group
ObjectGUID         : 8bccb1f2-713a-400f-979a-6231dbde7fa9
SamAccountName     : GClientes
SID                : S-1-5-21-2858253806-180064856-358215610-1223

PS C:\Users\Administrador>
```

Get-adgroup -filter {name -like '*GClientes*'}

También podemos emplear doble condición de filtrado.



```
Administrador: Windows PowerShell ISE
Archivo Editar Ver Herramientas Depurar Complementos Ayuda
Script
PS C:\Users\Administrador> Get-adgroup -filter {name -like '*Gdireccion*' -or name -eq 'GClientes'}

DistinguishedName : CN=GDireccion,OU=Direccion,DC=santiago,DC=local
GroupCategory      : Security
GroupScope         : Global
Name               : GDireccion
ObjectClass        : group
ObjectGUID         : af8ce765-c618-4d06-9406-1160dd737687
SamAccountName     : GDireccion
SID                : S-1-5-21-2858253806-180064856-358215610-1169

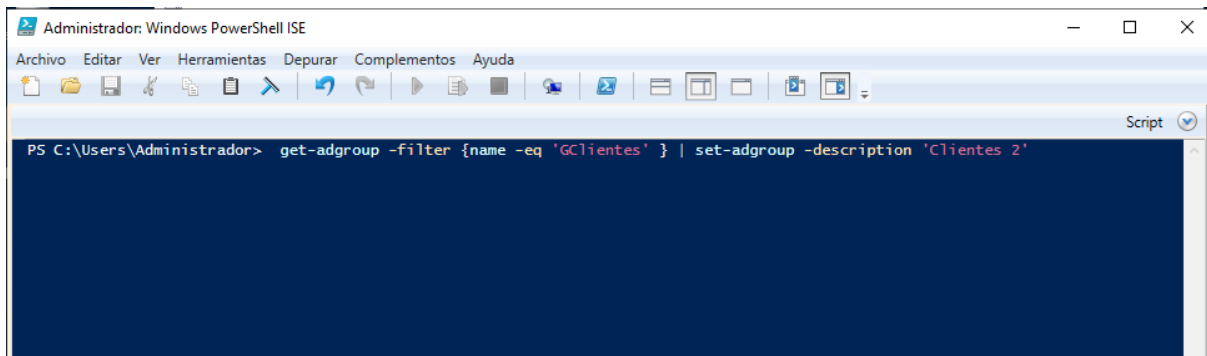
DistinguishedName : CN=GClientes,OU=Clientes,DC=santiago,DC=local
GroupCategory      : Security
GroupScope         : DomainLocal
Name               : GClientes
ObjectClass        : group
ObjectGUID         : 8bccb1f2-713a-400f-979a-6231dbde7fa9
SamAccountName     : GClientes
SID                : S-1-5-21-2858253806-180064856-358215610-1223

PS C:\Users\Administrador>
```

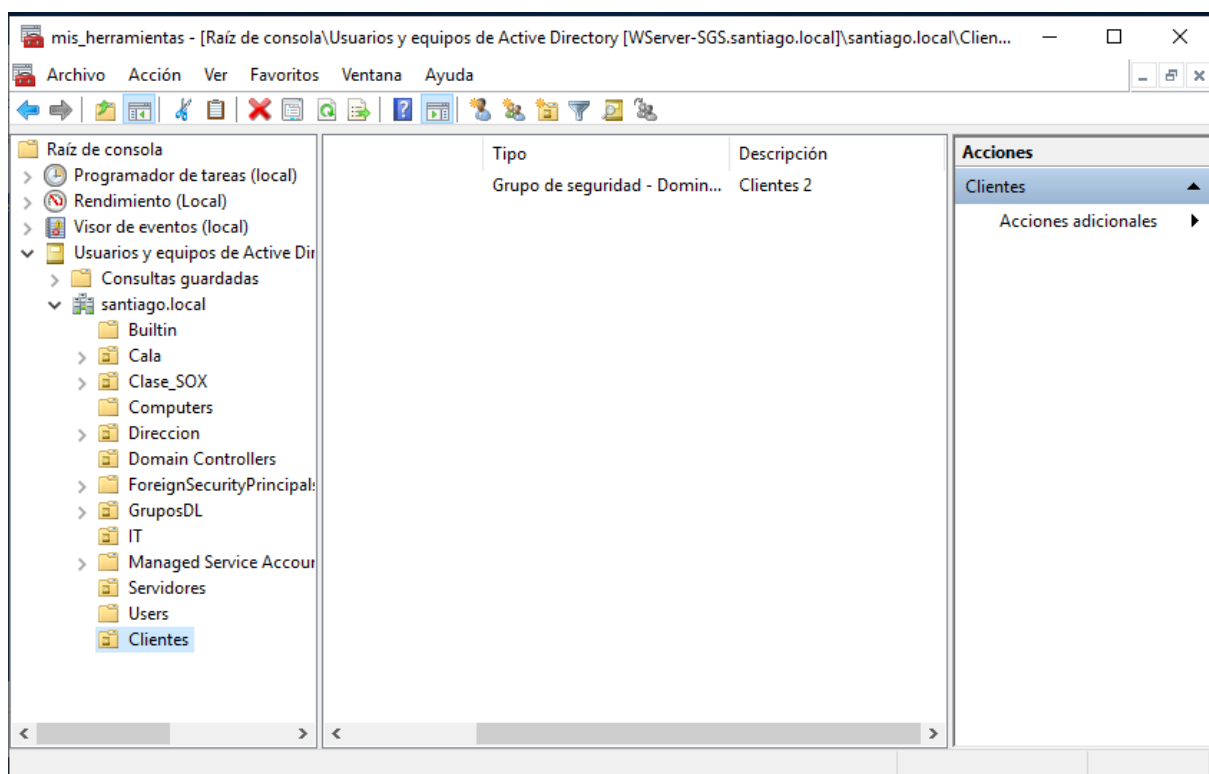
04- Modificar la información de un grupo

EL procedimiento es muy similar al que empleamos para modificar parámetros de los grupos de equipos o usuarios.

```
get-adgroup -filter {name -eq 'GClientes' } | set-adgroup -description  
'Clientes 2'
```



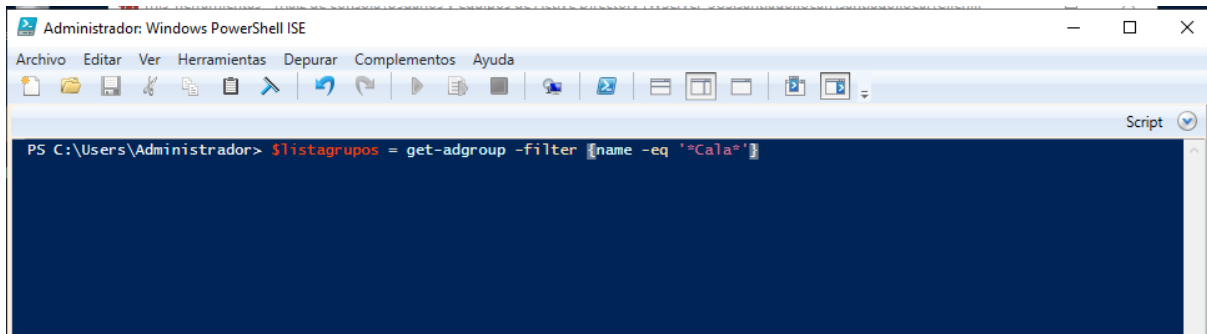
Primero filtramos los grupos para centrarnos en "GClientes", una vez en ellos mediante un pipeline ejecutamos el comando "set-adgroup" para añadir en este caso al campo "descripción" el string "clientes 2".



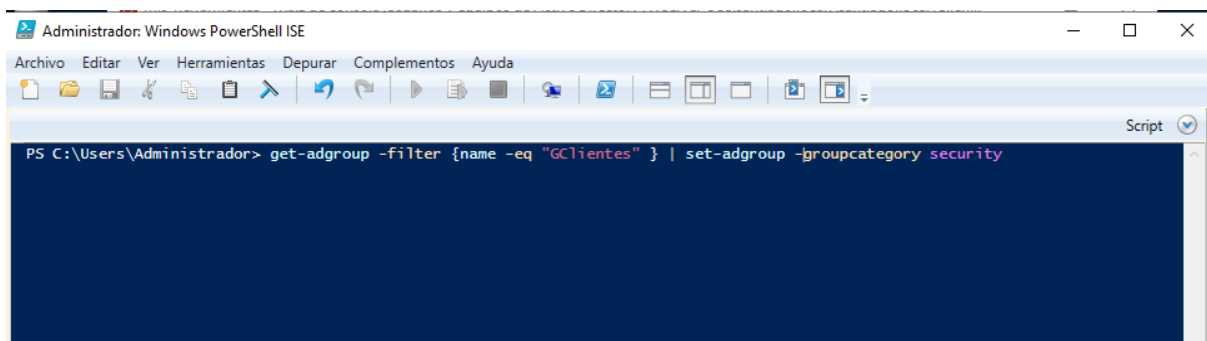
04- Modificar la información de un grupo

[Continuación]

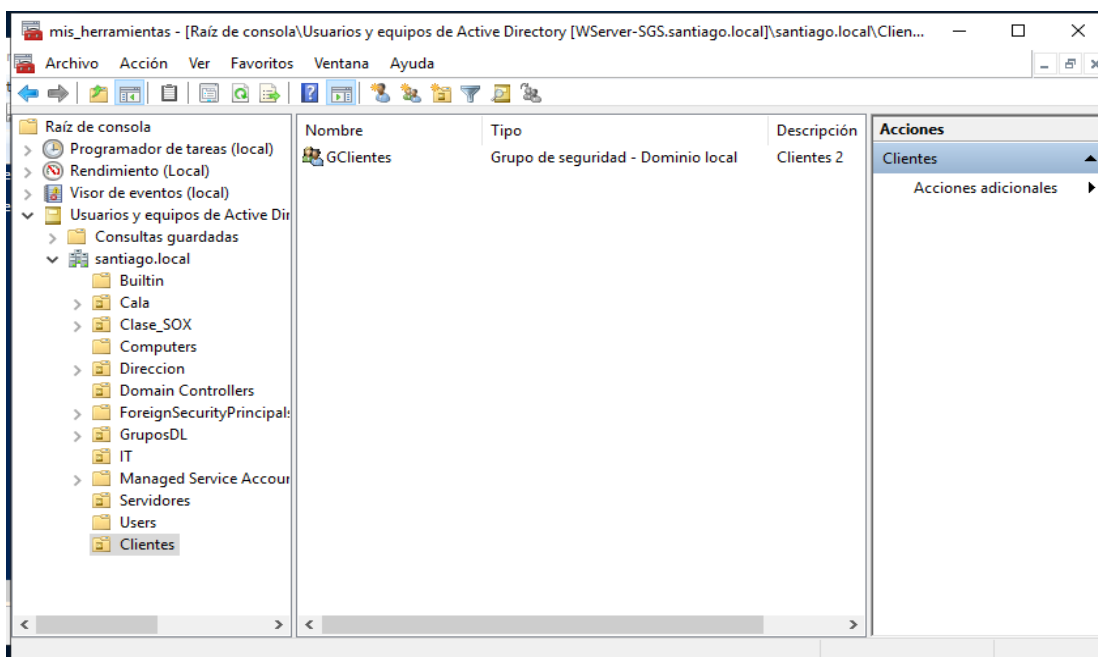
Del mismo modo podemos aplicar cambios a varios grupos desde el mismo comando, ya sea ampliando el filtro de búsqueda **get-adgroup** o manejando los grupos desde una variable de texto.



Del mismo modo podemos cambiar el tipo de grupo (security o distribution) mediante el parametro **-groupcategory**

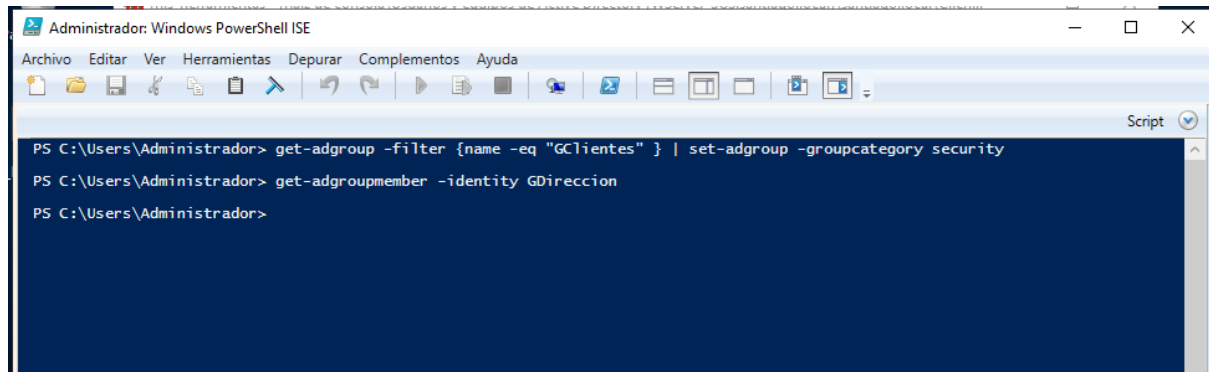


Aquí podemos apreciar como el grupo GClientes a pasado a ser de la categoría "security".



05- Ver miembros de un grupo

Mediante el cmdlet podemos listar los miembros de un grupo, para ello tenemos que emplear el comando `get-adgroupmember`, especificando mediante el parametro `-identity` el nombre del grupo.



```
PS C:\Users\Administrador> get-adgroup -filter {name -eq "GClientes" } | set-adgroup -groupcategory security
PS C:\Users\Administrador> get-adgroupmember -identity GDireccion
PS C:\Users\Administrador>
```

En este caso hemos empleado el comando:

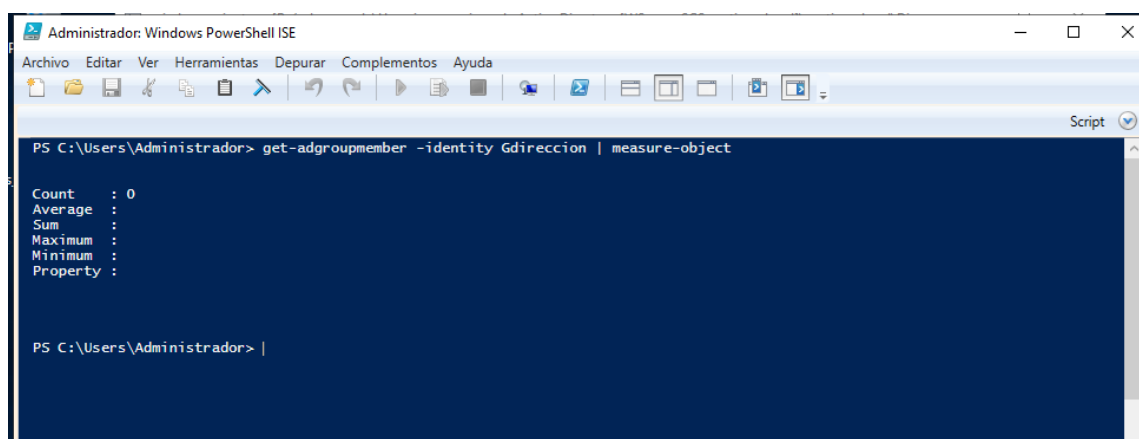
`get-adgroupmember -identity GDireccion`

Igualmente este comando nos mostrará en pantalla (en caso de tener miembros el mismo) los datos de los integrantes (distinguishedname, name, objectclass,objectguid,samaccountname y sid)

Podemos así mismo mostrar el número de miembros de un grupo mediante:

`get-adgroupmember -identity Gdireccion | measure-object`

Identificando el grupo mediante el string del parámetro `"-identity"`, pipeline después, el `"measure-object"` nos indicaria el tamaño en registros del mismo.



```
PS C:\Users\Administrador> get-adgroupmember -identity Gdireccion | measure-object

Count      : 0
Average    :
Sum        :
Maximum    :
Minimum    :
Property   :

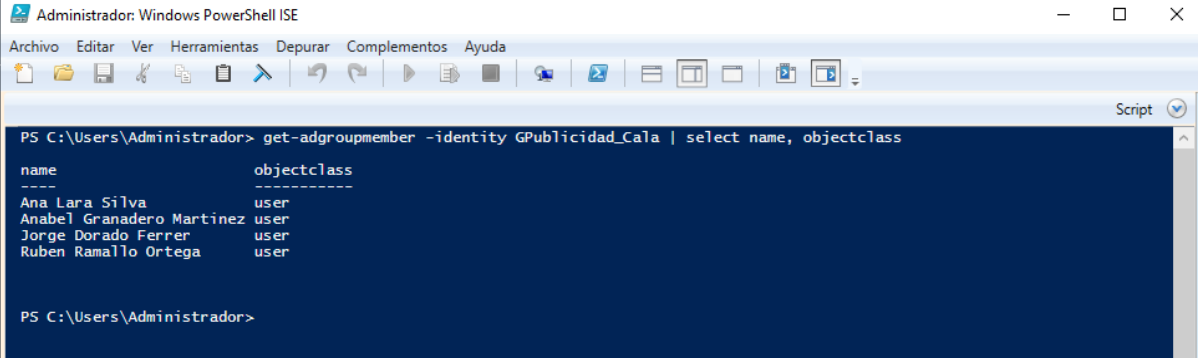
PS C:\Users\Administrador> |
```

Puesto que este grupo en particular `"Gdireccion"` no cuenta con ningún miembro, el count nos viene indicado a `"0"`.

05- Ver miembros de un grupo

[Continuación]

También podemos visualizar los datos en formato de tabla, en este caso consultaremos el grupo "GPublicidadcala".



```
PS C:\Users\Administrador> get-adgroupmember -identity GPublicidad_Cala | select name, objectclass
```

name	objectclass
Ana Lara Silva	user
Anabel Granadero Martinez	user
Jorge Dorado Ferrer	user
Ruben Ramallo Ortega	user

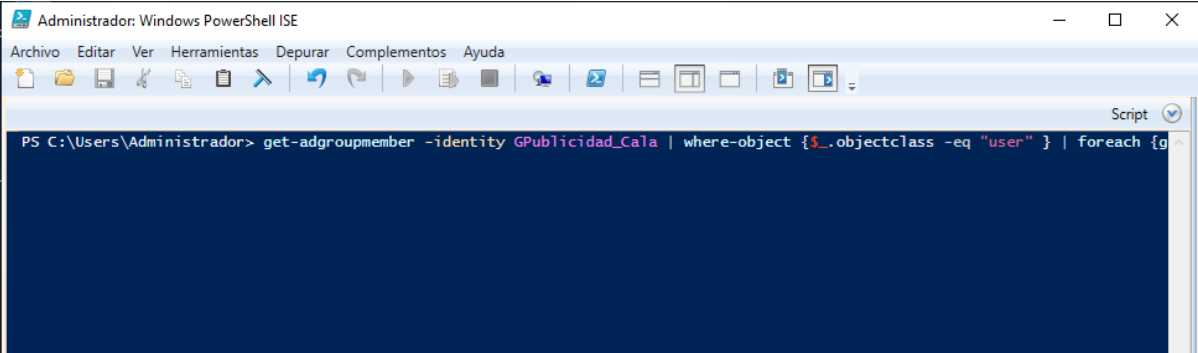
```
PS C:\Users\Administrador>
```

get-adgroupmember -identity GPublicidadCala | select name, objectclass

En esta consulta hemos seleccionado la "identity" del grupo "GPublicidadcala" y pipeline después seleccionado para la muestra el parámetro "objectclass".

Si lo necesitamos, también podríamos ampliar la cantidad de datos mostrados, realizando filtrados adicionales.

get-adgroupmember -identity GPublicidadCala | where-object {\$_.objectclass -eq "user"} | foreach {get-aduser -identity \$_} | select name, enabled, distinguishedname, emailaddress



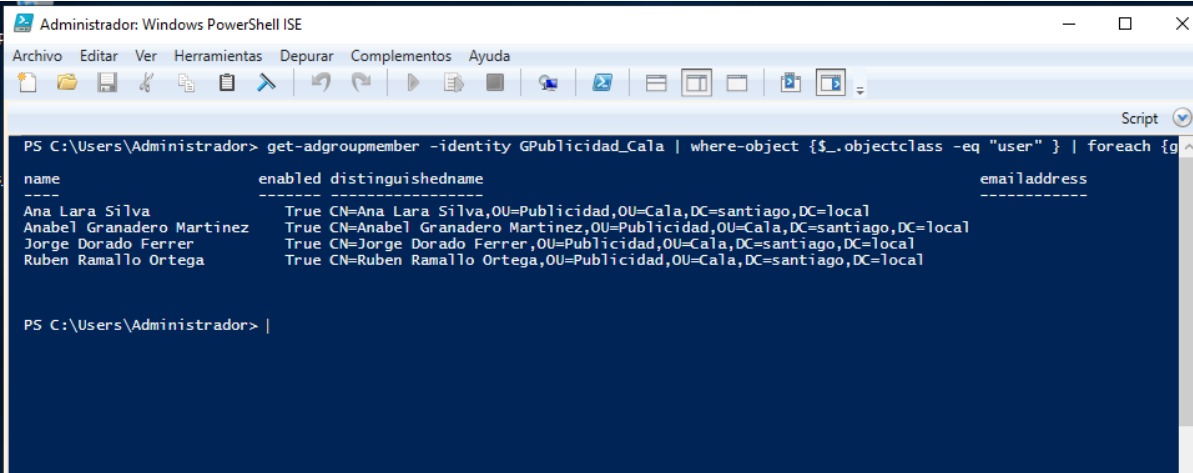
```
PS C:\Users\Administrador> get-adgroupmember -identity GPublicidad_Cala | where-object {$_.objectclass -eq "user"} | foreach {get-aduser -identity $_} | select name, enabled, distinguishedname, emailaddress
```

En este caso estaríamos seleccionando la identity "GPublicidadCala", pipeline después, el objeto dentro de esa entidad denominado "user", pipeline después, solicitamos ver los datos de "identity", "select name", que estén "enabled", junto al "distinguishedname" y su "emailaddress"

05- Ver miembros de un grupo

[Continuación]

Siendo este el resultado de la consulta con multiparametros.



```
PS C:\Users\Administrador> get-adgroupmember -identity GPublicidad_Cala | where-object {$_.objectclass -eq "user" } | foreach {g
```

name	enabled	distinguishedname	emailaddress
Ana Lara Silva	True	CN=Ana Lara Silva,OU=Publicidad,OU=Cala,DC=santiago,DC=local	
Anabel Granadero Martinez	True	CN=Anabel Granadero Martinez,OU=Publicidad,OU=Cala,DC=santiago,DC=local	
Jorge Dorado Ferrer	True	CN=Jorge Dorado Ferrer,OU=Publicidad,OU=Cala,DC=santiago,DC=local	
Ruben Ramallo Ortega	True	CN=Ruben Ramallo Ortega,OU=Publicidad,OU=Cala,DC=santiago,DC=local	

```
PS C:\Users\Administrador> |
```

06- Añadir miembros a un grupo

[capturas no disponibles]

Para añadir miembros usaremos el cmdlet **add-adgroupmember**

```
add-adgroupmember -identity "GClientes" -members tsanchez
```

Seleccionamos la "-identity" "GClientes" y añadimos al parametro "-members" tsanchez.

Mediante este mismo cmdlet podríamos añadir más de un miembro en una sola línea de comando, para ello tendríamos que separar cada uno de los miembros mediante una coma.

```
add-adgroupmember -identity "GClientes" -members mojeda,surrutia
```

El comando superior, añadiría los usuarios "mojeda" y "surrutia" al grupo "GClientes".

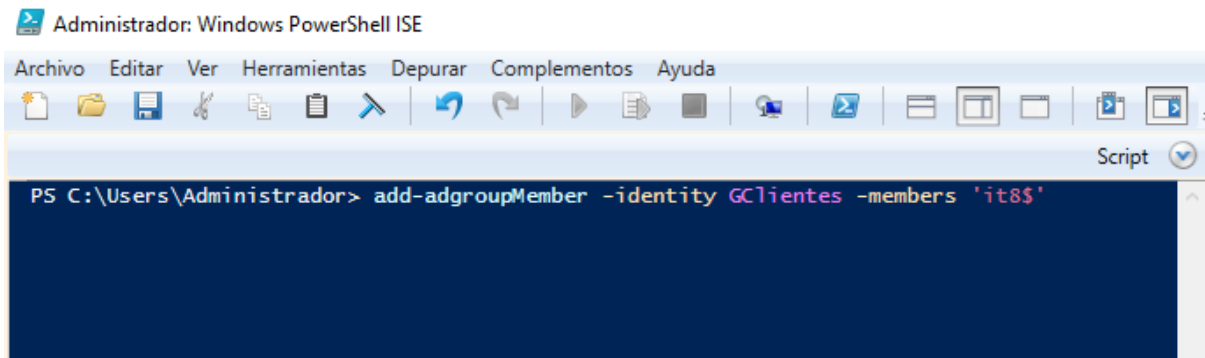
07- Añadir ordenadores a un grupo

El procedimiento es exactamente igual que el empleado anteriormente, pero en esta ocasión se le añade el carácter "\$" que es el que define a active directory que estamos tratando con una máquina.

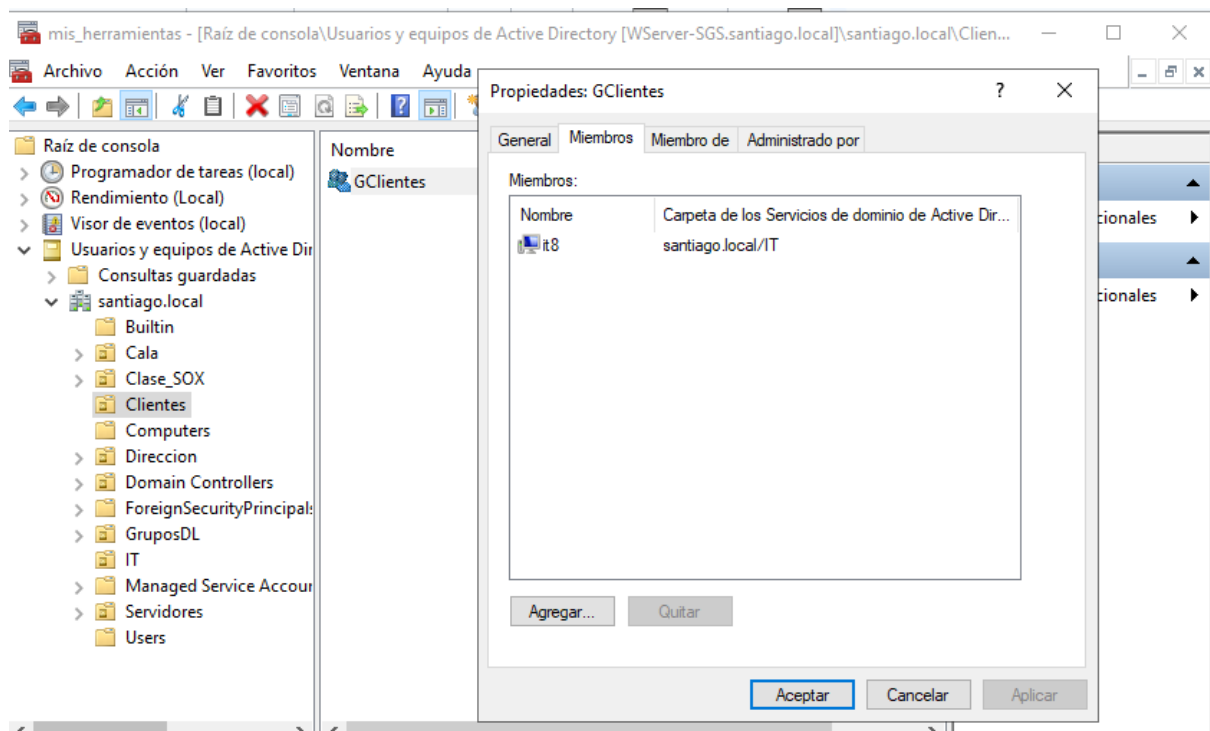
```
add-adgroupMember -identity GClientes -members 'it8$'
```

07- Añadir ordenadores a un grupo

[Continuación]



Y tras actualizar el active directory, podremos verificar la creación de la nueva máquina en GClientes.



08- Eliminar miembros de un grupo

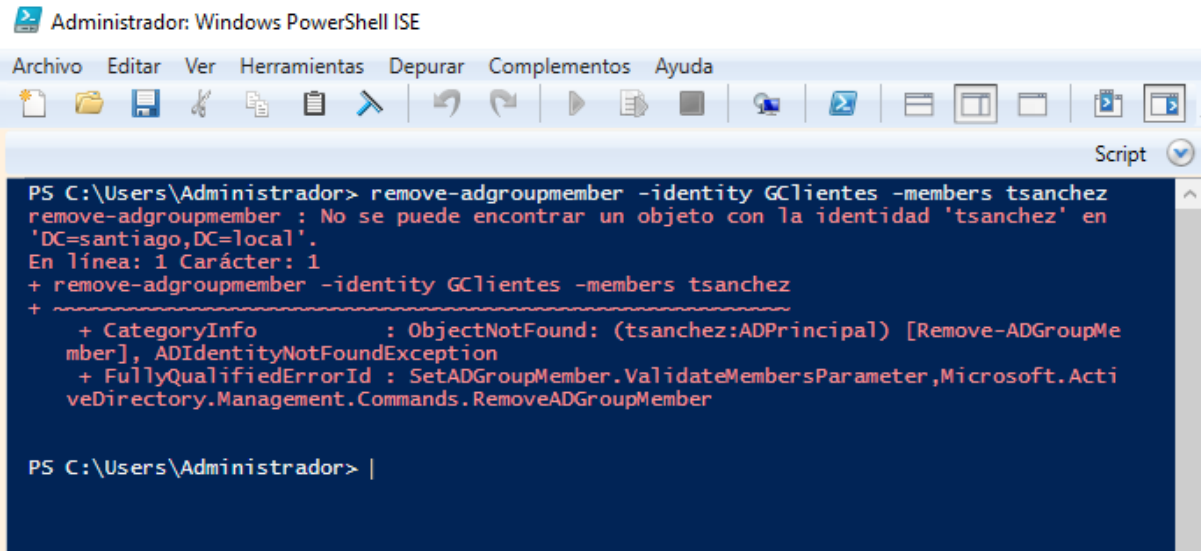
Una de las tareas que podemos realizar de manera habitual (siempre que sea necesario) es el eliminar miembros u objetos que ya no estén activos. Para ello empleamos el cmdlet **remove-adgroupmember**.

Dentro de este cmdlet, tenemos dos parámetros; **"-identity"** con el que identificamos el nombre del grupo y **"-members"** donde especificamos que miembros hay que remover.

remove-adgroupmember -identity GClientes -members tsanchez

08- Eliminar miembros de un grupo

[Continuación]



Administrador: Windows PowerShell ISE

Archivo Editar Ver Herramientas Depurar Complementos Ayuda

Script

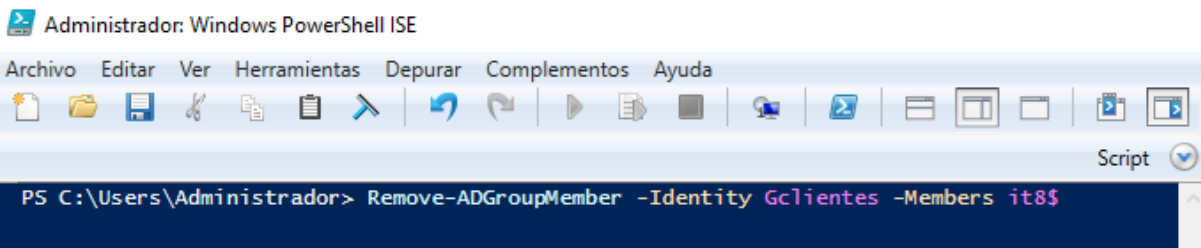
```
PS C:\Users\Administrador> remove-adgroupmember -identity GClientes -members tsanchez
remove-adgroupmember : No se puede encontrar un objeto con la identidad 'tsanchez' en
'DC=santiago,DC=local'.
En línea: 1 Carácter: 1
+ remove-adgroupmember -identity GClientes -members tsanchez
+ ~~~~~
+ CategoryInfo          : ObjectNotFound: (tsanchez:ADPrincipal) [Remove-ADGroupMe
mber], ADIdentityNotFoundException
+ FullyQualifiedErrorId : SetADGroupMember.ValidateMembersParameter,Microsoft.Acti
veDirectory.Management.Commands.RemoveADGroupMember

PS C:\Users\Administrador> |
```

Este cmdlet de nuevo ejecuta una acción considerada delicada, por lo que siempre nos pedirá confirmación. Podemos evitar este diálogo añadiendo el parámetro **-confirm:\$false** al final del mismo. Por cuestiones de seguridad esta práctica no está recomendada.

Para eliminar un ordenador, como en el caso de añadirlo tenemos que añadir el carácter "\$" a su correspondiente "samaccountname".

Remove-ADGroupMember -Identity Gclientes -Members it8\$

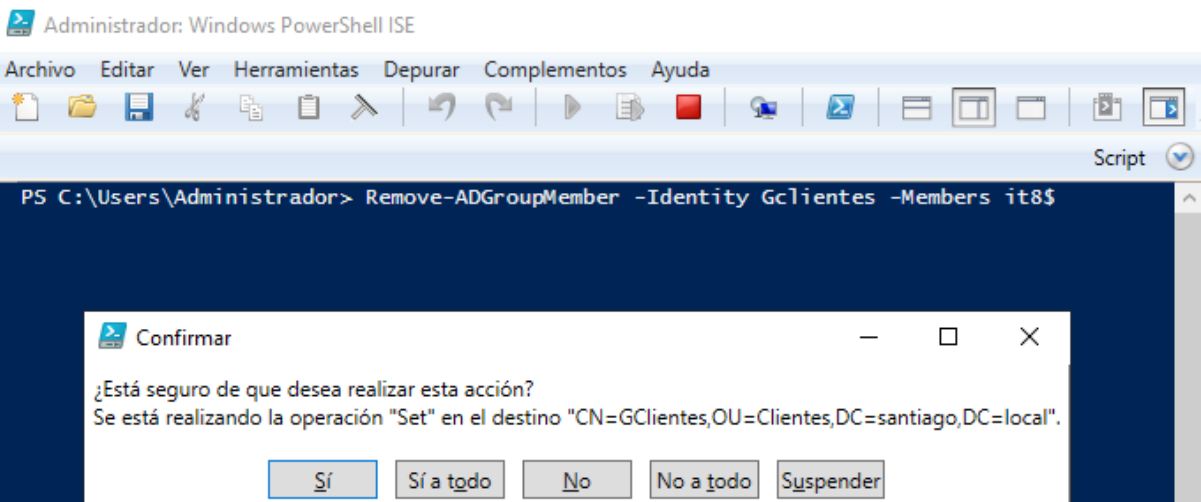


Administrador: Windows PowerShell ISE

Archivo Editar Ver Herramientas Depurar Complementos Ayuda

Script

```
PS C:\Users\Administrador> Remove-ADGroupMember -Identity Gclientes -Members it8$
```



Administrador: Windows PowerShell ISE

Archivo Editar Ver Herramientas Depurar Complementos Ayuda

Script

```
PS C:\Users\Administrador> Remove-ADGroupMember -Identity Gclientes -Members it8$
```

Confirmar

¿Está seguro de que desea realizar esta acción?
Se está realizando la operación "Set" en el destino "CN=GClientes,OU=Clientes,DC=santiago,DC=local".

Sí Sí a todo No No a todo Suspende

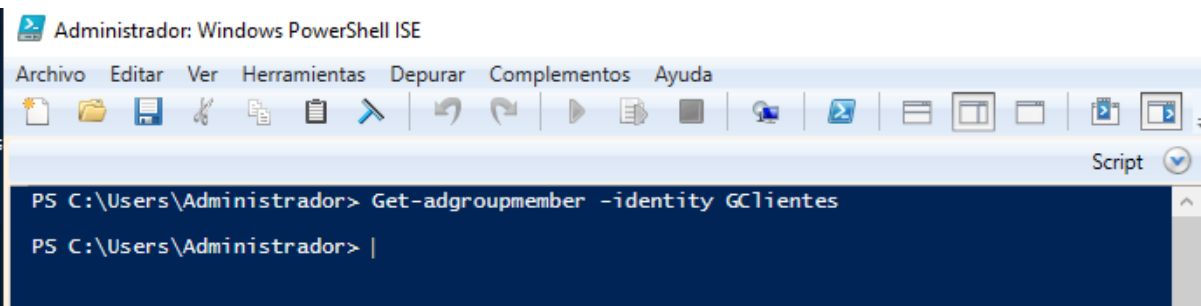
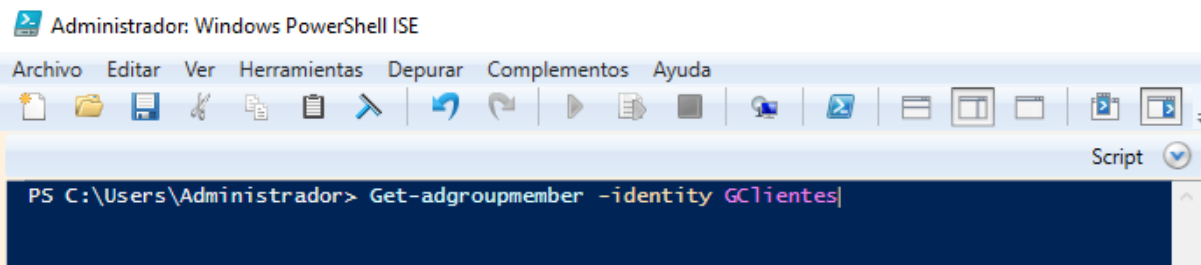
09- Borrar un grupo

Si necesitamos borrar un grupo, podemos realizarlo fácilmente desde el Powershell, eso sí tenemos que asegurarnos de que dicho grupo esté completamente vacío.

Para ello primero seleccionamos el grupo:

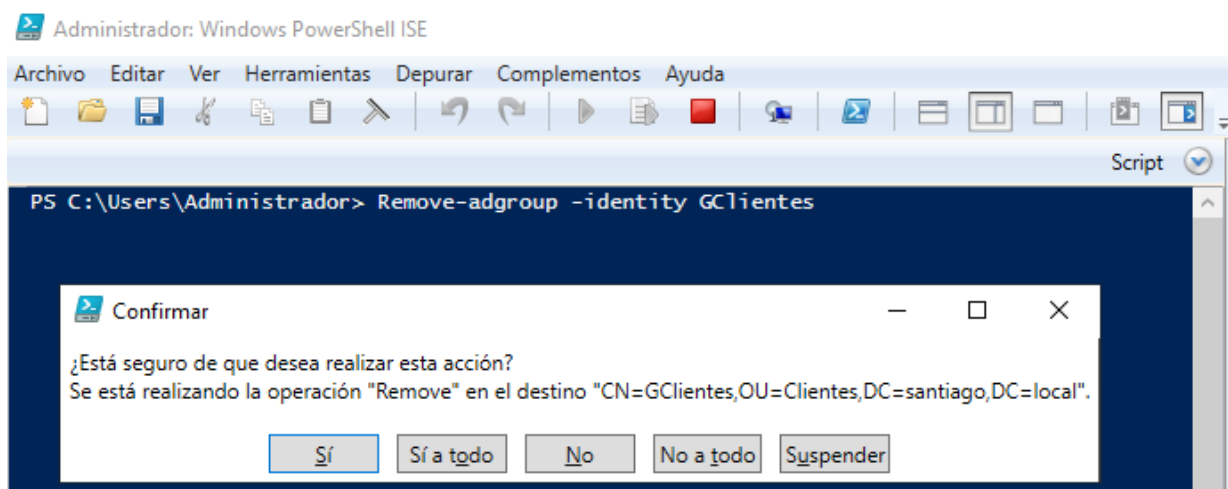
Get-adgroupmember -identity GClientes

Mediante la orden "Get" solicitamos ver los miembros del grupo "identity" "Gclientes".



Una vez hemos verificado que el grupo está completamente vacío, procedemos a introducir el cmdlet para su borrado.

Remove-adgroup -identity GClientes

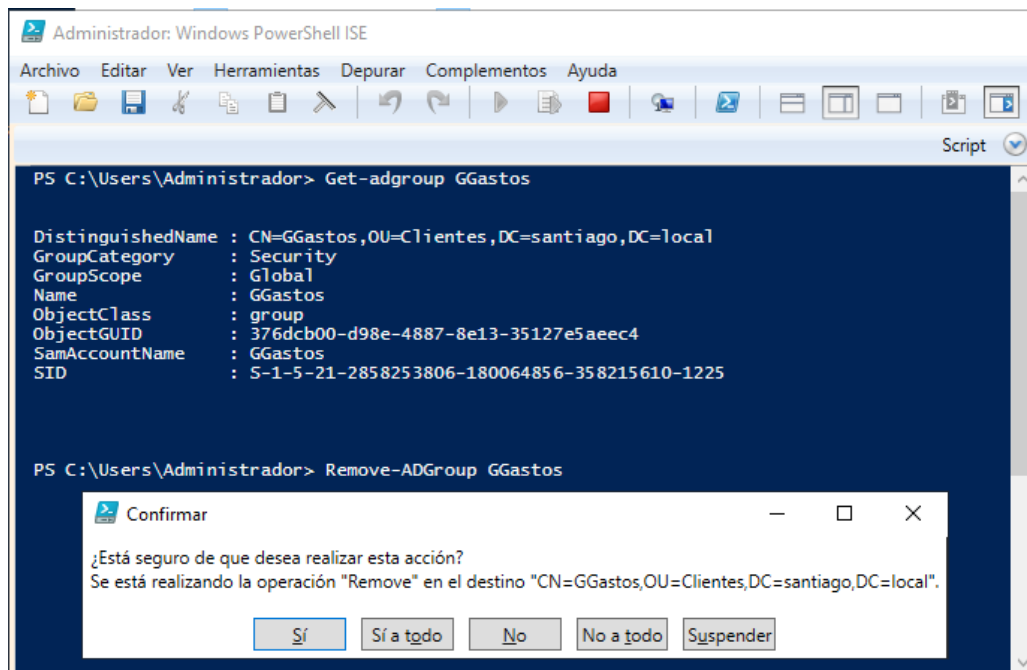
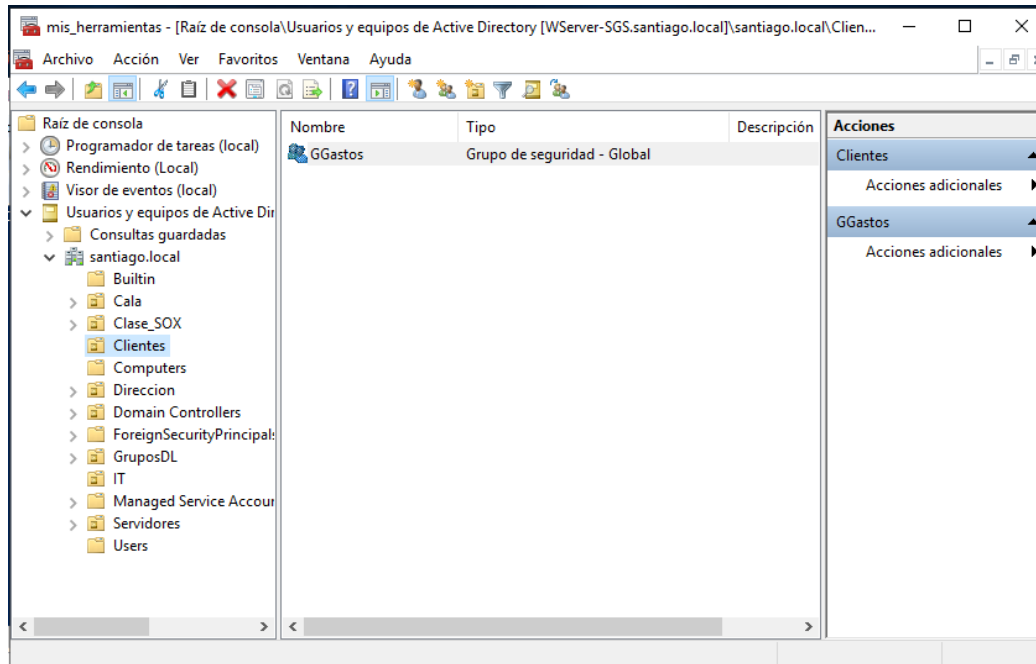


09- Borrar un grupo

[Continuación]

Como hemos hecho anteriormente podemos pasarle el grupo a eliminar a este comando mediante una búsqueda previa empleando pipelines.

Get-adgroup -filter "name -eq 'GGastos' | remove-adgroup

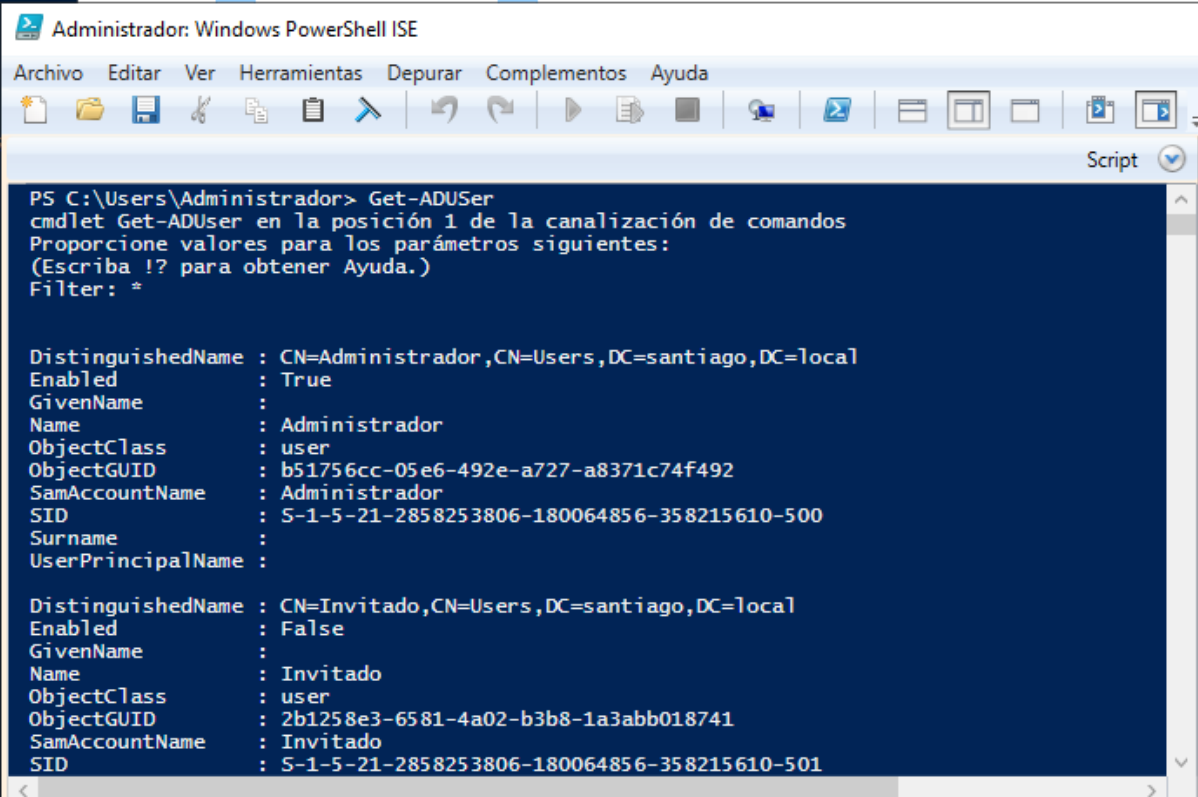


En este caso en particular, hemos tenido que crear el grupo, para posteriormente poder eliminarlo en el ejemplo.

10- Información proporcionada por Get-ADUser

DistinguishedName	Proporcionará el nombre canónico completo del usuario.
Habilitado	Mostrará si la cuenta de usuario está habilitada o no.
GivenName	Es el nombre legible por los humanos que es generalmente el nombre y apellido del usuario.
Clase Objeto	Es el tipo de usuario que generalmente es «usuario».
ObjetoGUID	Es la identificación única del usuario u objeto actual.
Nombre de la cuenta SAM	Es el nombre de la cuenta SAM del usuario.

También podemos visualizar todos los usuarios haciendo uso del carácter especial (*)



```
Administrador: Windows PowerShell ISE
Archivo Editar Ver Herramientas Depurar Complementos Ayuda
PS C:\Users\Administrador> Get-ADUser
cmdlet Get-ADUser en la posición 1 de la canalización de comandos
Proporcione valores para los parámetros siguientes:
(Escriba !? para obtener Ayuda.)
Filter: *

DistinguishedName : CN=Administrador,CN=Users,DC=santiago,DC=local
Enabled           : True
GivenName         :
Name              : Administrador
ObjectClass       : user
ObjectGUID        : b51756cc-05e6-492e-a727-a8371c74f492
SamAccountName    : Administrador
SID               : S-1-5-21-2858253806-180064856-358215610-500
Surname           :
UserPrincipalName :

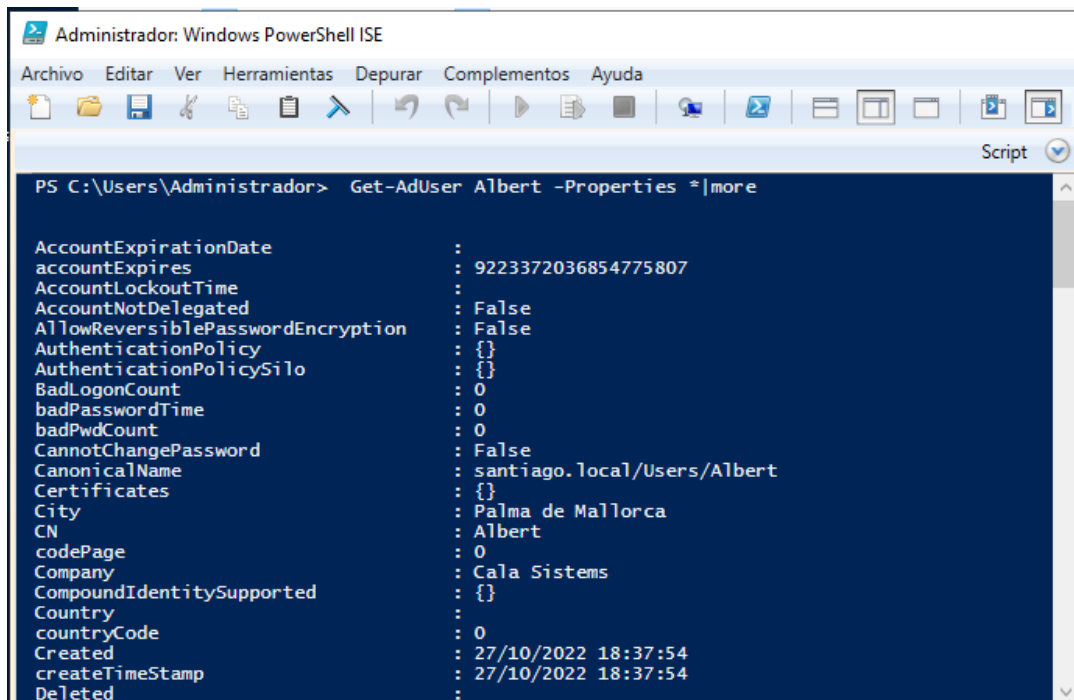
DistinguishedName : CN=Invitado,CN=Users,DC=santiago,DC=local
Enabled           : False
GivenName         :
Name              : Invitado
ObjectClass       : user
ObjectGUID        : 2b1258e3-6581-4a02-b3b8-1a3abb018741
SamAccountName    : Invitado
SID               : S-1-5-21-2858253806-180064856-358215610-501
```

11- Mostrar toda la información de un usuario específico

Para ello empleamos el siguiente cmdlet.

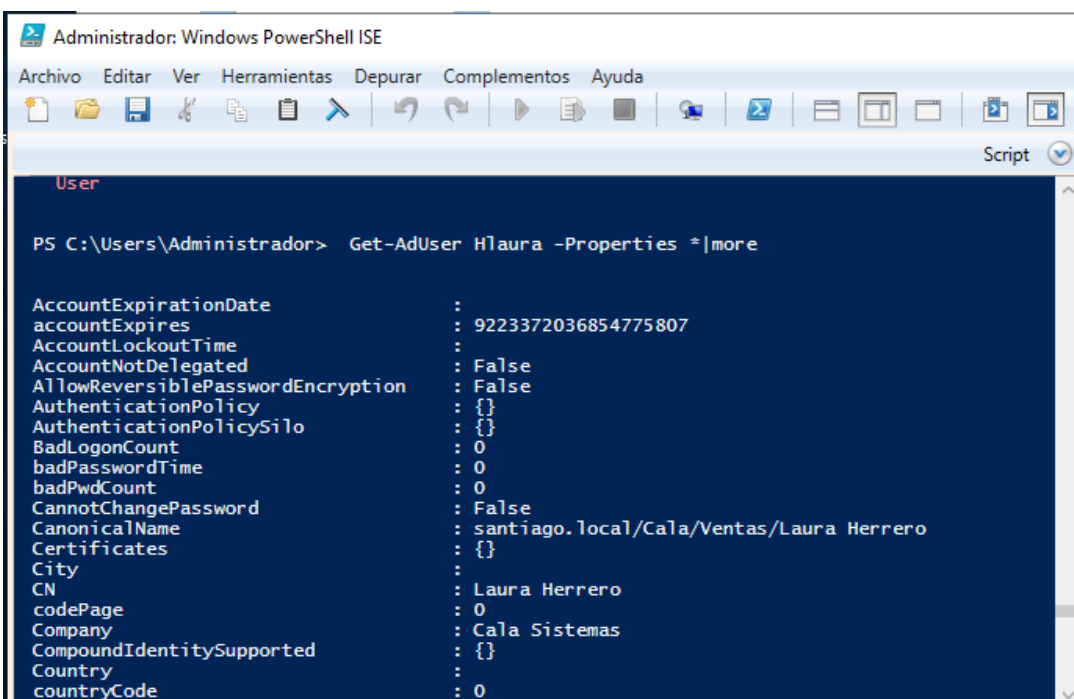
```
PS C:\Users\Administrador> Get-AdUser Albert -Properties *|more
```

Donde especificamos mostrar los datos del usuario "Albert" con todas sus propiedades.



```
Administrador: Windows PowerShell ISE
Archivo  Editar  Ver  Herramientas  Depurar  Complementos  Ayuda
Script
PS C:\Users\Administrador> Get-AdUser Albert -Properties *|more
AccountExpirationDate      :
accountExpires             : 9223372036854775807
AccountLockoutTime         :
AccountNotDelegated        : False
AllowReversiblePasswordEncryption : False
AuthenticationPolicy       : {}
AuthenticationPolicySilo   : {}
BadLogonCount              : 0
badPasswordTime            : 0
badPwdCount                : 0
CannotChangePassword       : False
CanonicalName              : santiago.local/Users/Albert
Certificates               : {}
City                       : Palma de Mallorca
CN                         : Albert
codePage                   : 0
Company                    : Cala Sistemas
CompoundIdentitySupported   : {}
Country                    :
countryCode                : 0
Created                    : 27/10/2022 18:37:54
createTimeStamp            : 27/10/2022 18:37:54
Deleted                    :
```

```
PS C:\Users\Administrador> Get-AdUser Hlaura -Properties *|more
```

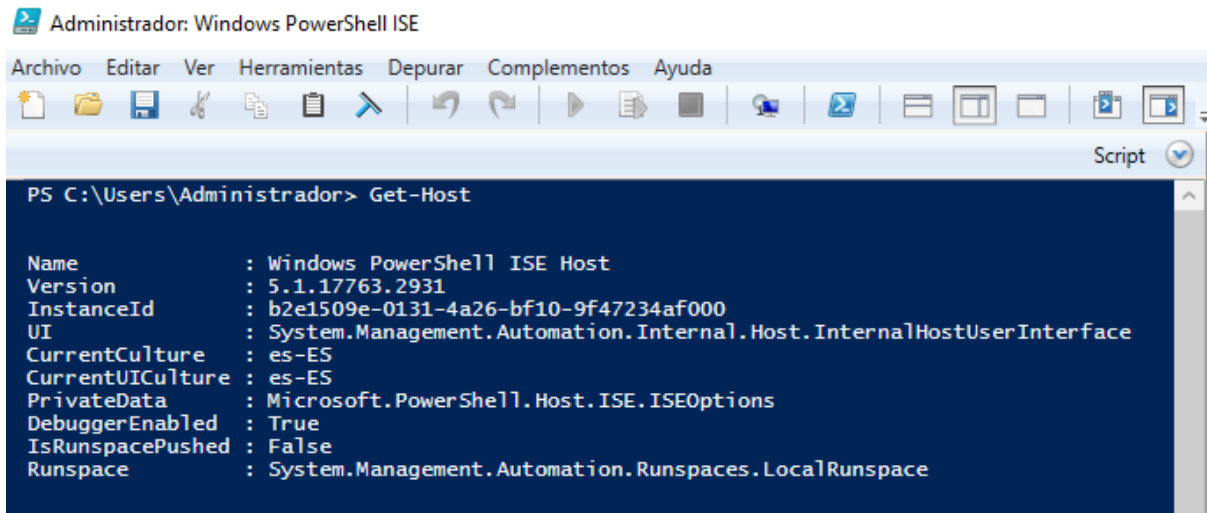


```
Administrador: Windows PowerShell ISE
Archivo  Editar  Ver  Herramientas  Depurar  Complementos  Ayuda
Script
User
PS C:\Users\Administrador> Get-AdUser Hlaura -Properties *|more
AccountExpirationDate      :
accountExpires             : 9223372036854775807
AccountLockoutTime         :
AccountNotDelegated        : False
AllowReversiblePasswordEncryption : False
AuthenticationPolicy       : {}
AuthenticationPolicySilo   : {}
BadLogonCount              : 0
badPasswordTime            : 0
badPwdCount                : 0
CannotChangePassword       : False
CanonicalName              : santiago.local/Cala/Ventas/Laura Herrero
Certificates               : {}
City                       :
CN                         : Laura Herrero
codePage                   : 0
Company                    : Cala Sistemas
CompoundIdentitySupported   : {}
Country                    :
countryCode                : 0
```


12- Verificación de la versión Powershell

Existen distintas versiones de PowerShell. Para identificar la que se está ejecutando podemos emplear diferentes instrucciones en el cmdlet.

Get-Host

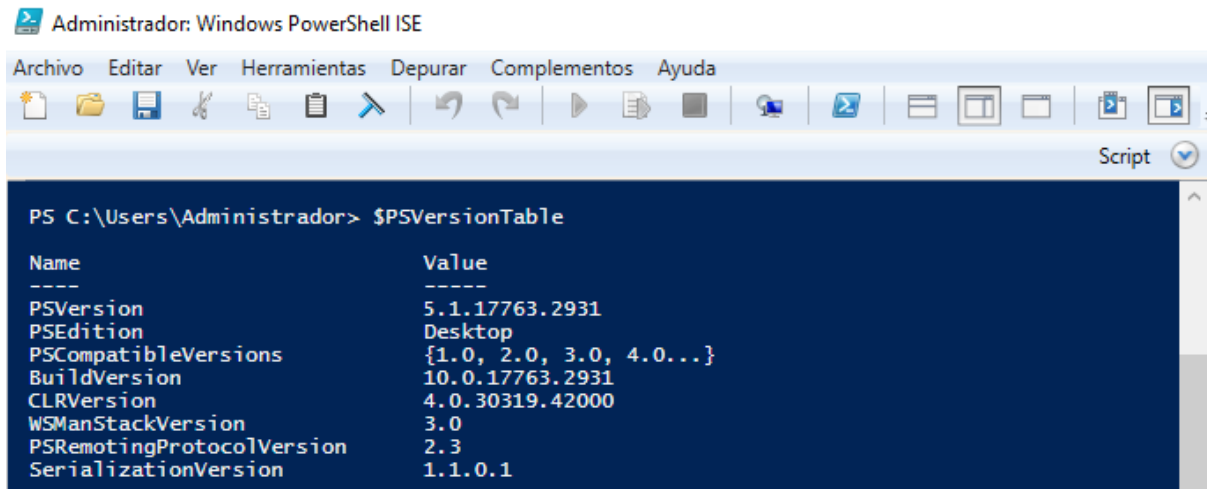


Administrador: Windows PowerShell ISE

```
PS C:\Users\Administrador> Get-Host
```

Name	: Windows PowerShell ISE Host
Version	: 5.1.17763.2931
InstanceId	: b2e1509e-0131-4a26-bf10-9f47234af000
UI	: System.Management.Automation.Internal.Host.InternalHostUserInterface
CurrentCulture	: es-ES
CurrentUICulture	: es-ES
PrivateData	: Microsoft.PowerShell.Host.ISE.ISEOptions
DebuggerEnabled	: True
IsRunspacePushed	: False
Runspace	: System.Management.Automation.Runspace.LocalRunspace

\$PSVersionTable

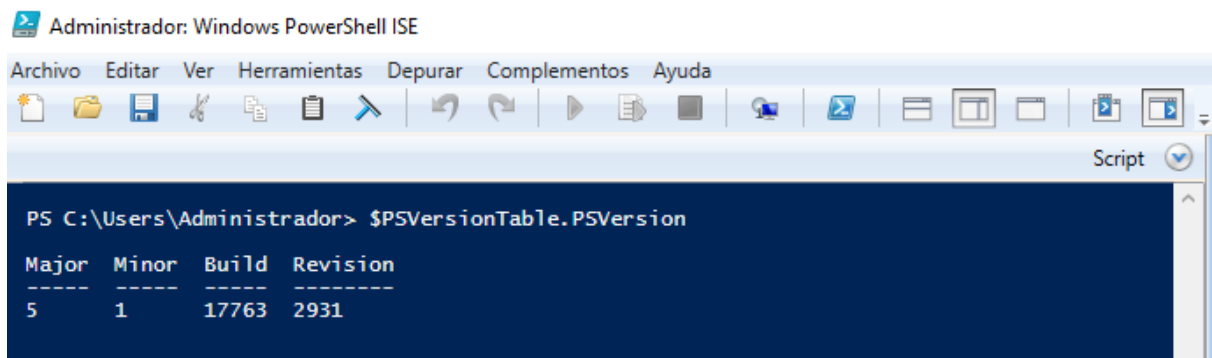


Administrador: Windows PowerShell ISE

```
PS C:\Users\Administrador> $PSVersionTable
```

Name	Value
PSVersion	5.1.17763.2931
PSEdition	Desktop
PSCompatibleVersions	{1.0, 2.0, 3.0, 4.0...}
BuildVersion	10.0.17763.2931
CLRVersion	4.0.30319.42000
WSManStackVersion	3.0
PSRemotingProtocolVersion	2.3
SerializationVersion	1.1.0.1

\$PSVersionTable.PSVersion



Administrador: Windows PowerShell ISE

```
PS C:\Users\Administrador> $PSVersionTable.PSVersion
```

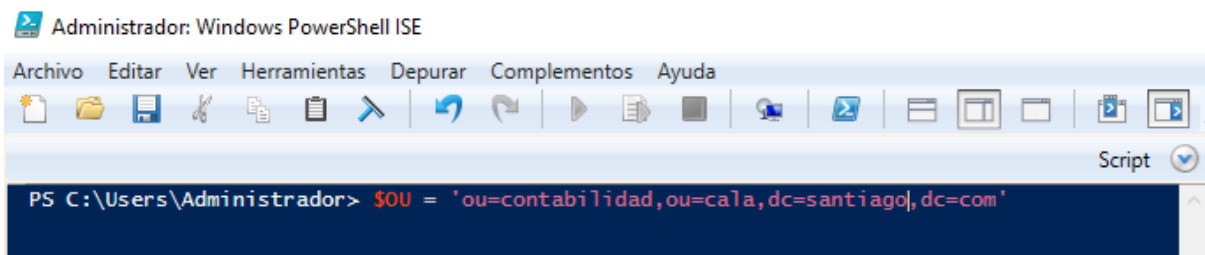
Major	Minor	Build	Revision
5	1	17763	2931

13- Listar usuarios de una UO concreta

Mediante el Powershell podemos listar los usuarios de una unidad organizativa concreta, para ello existen diferentes métodos, siendo uno de los más rápidos el que incluye el uso de una variante de texto. Variante en la que almacenamos toda la información de los usuarios de la UO seleccionada.

\$OU = 'ou=contabilidad,ou=cala,dc=santiago,dc=com'

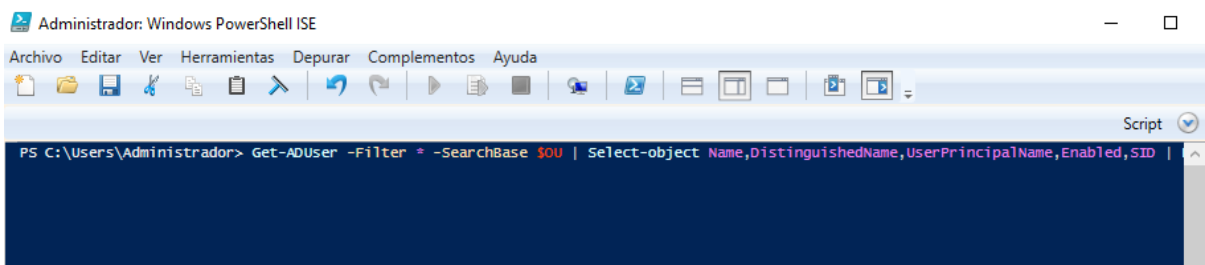
En este caso estaríamos seleccionado la unidad organizativa "contabilidad" incluida en la unión organizativa "cala" de nuestro dominio.



A Continuación, introducimos el siguiente comando.

**PS C:\Users\Administrador> Get-ADUser -Filter * -SearchBase \$OU |
Select-objectName,DistinguishedName,UserPrincipalName,Enabled,SID |
Export-CSV "C:\UsuariosOU2.CSV" -NoTypeInformation**

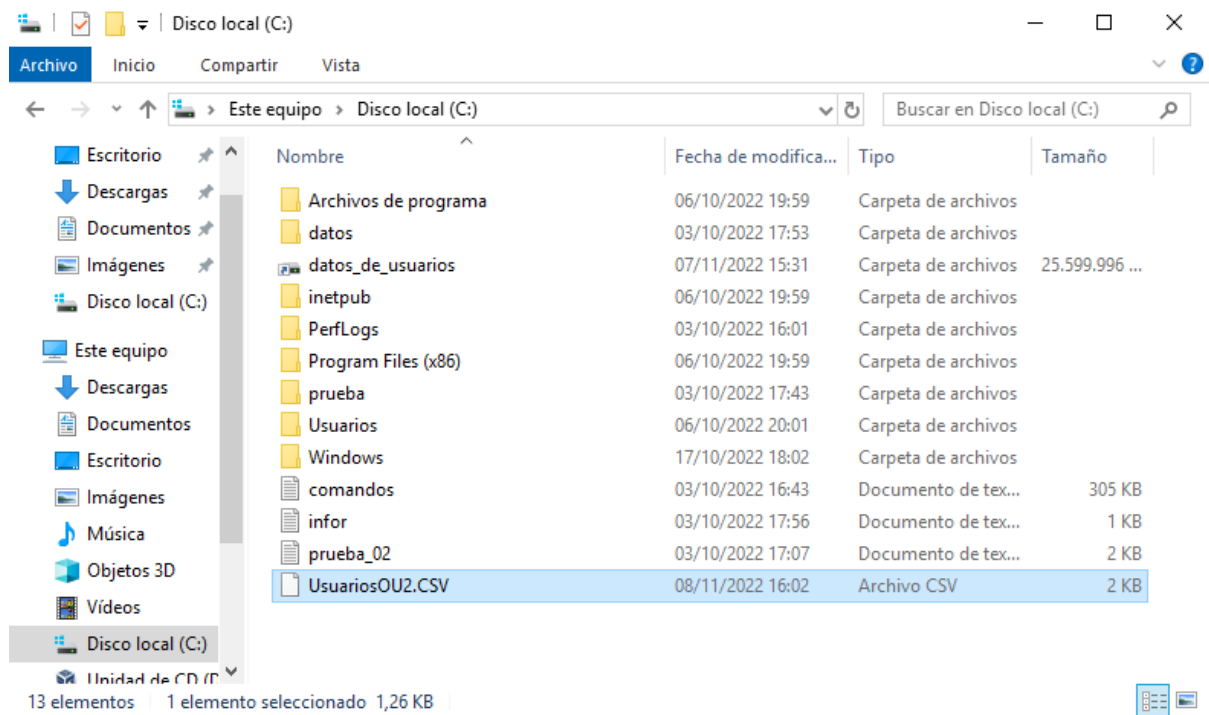
En el solicitamos que nos muestre todos los campos de la base almacenada en la variable de texto, y los campos que deseamos ver (-objectname,distinguishedname,userprincipalname,enabled,SID) todo esto exportado a una archivo de extensión CSV y la ruta en nuestro sistema donde queremos que esté situado el archivo.



13- Listar usuarios de una UO concreta

[Continuación]

Tras este comando, podemos revisar la ruta que hemos introducido para hospedar el archivo .CSV



Y al abrir dicho archivo con el notepad, visualizar todos los registros de la UO exportada.



Estas son las diferentes opciones de las que disponemos.

Get-ADUser	Hace una consulta sobre los usuarios de AD DS.
SearchBase	Permite cambiar el alcance de la consulta para mostrar solo los usuarios que nos interesan, en este caso, los de una OU en concreto.
Select-object	Permite seleccionar que campos queremos que se muestran en la consulta que estamos haciendo. Para saber los campos, se recomienda lanzar el comando original sin modificadores para ver qué información se puede obtener. "Get-ADUser -Filter *".
Export-CSV	Permite extraer la información a un fichero en formato CSV separado con comas (,) que permitirá ser usado por scripts posteriores o ser abierto con un Excel para su visualización.
NoTypeInfoInformation	Este modificador, elimina la primera línea del fichero CSV, esta línea es informativa sobre la ejecución del comando y puede entorpecer el uso del fichero en caso de necesitarlo para la ejecución de otro script.

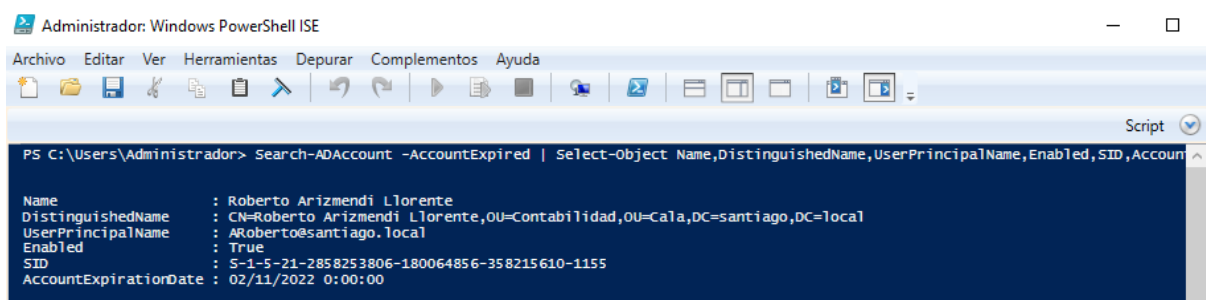
14- Listar cuentas de usuario expiradas

Para emplear esta consulta, tenemos que utilizar el comando "**search-ADAccount**" puesto que a diferencia de "Get-ADUser" sí que permite hacer la consulta sobre la fecha de expiración de los accesos.

Al Usuario Roberto Arizmendi Llorente, le caduco la cuenta el día 01/11/22.

El comando sería el siguiente.

```
Search-ADAccount -AccountExpired | Select-Object  
Name,DistinguishedName,UserPrincipalName,Enabled,SID,AccountExpirationDate
```

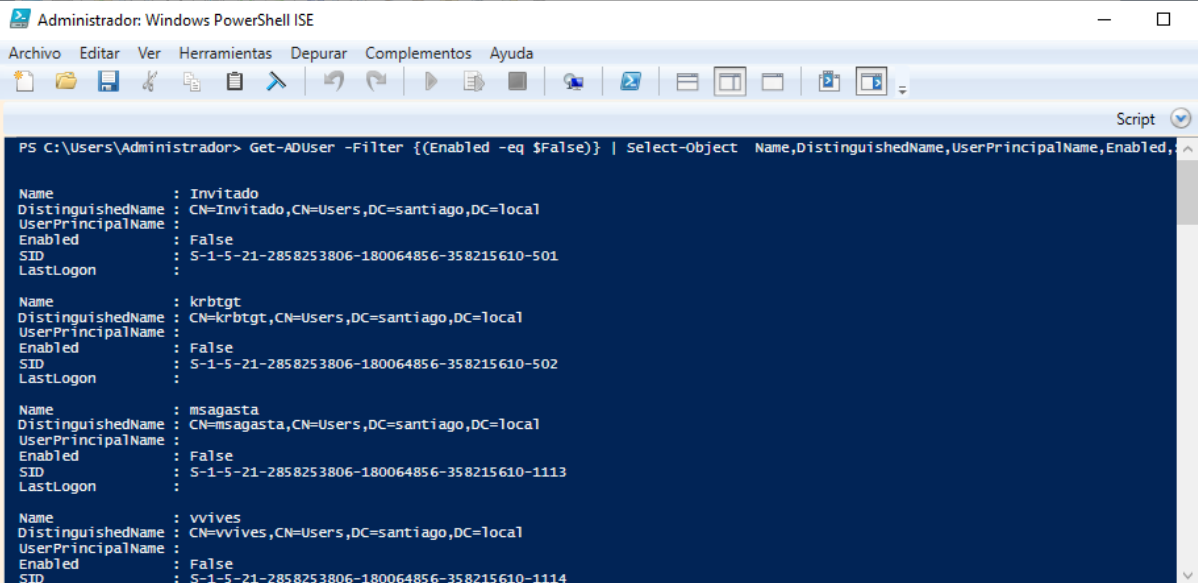


```
Administrador: Windows PowerShell ISE
Archivo Editar Ver Herramientas Depurar Complementos Ayuda
PS C:\Users\Administrador> Search-ADAccount -AccountExpired | Select-Object Name,DistinguishedName,UserPrincipalName,Enabled,SID,AccountExpirationDate
Name : Roberto Arizmendi Llorente
DistinguishedName : CN=Roberto Arizmendi Llorente,OU=Contabilidad,OU=Ca1a,DC=santiago,DC=local
UserPrincipalName : ARoberto@santiago.local
Enabled : True
SID : S-1-5-21-2858253806-180064856-358215610-1155
AccountExpirationDate : 02/11/2022 0:00:00
```

15- Listar cuentas de usuario deshabilitadas

Las cuentas deshabilitadas se listan mediante el comando “**Get-ADUser**” y consultando la propiedad “**Enabled**” está como un falso booleano, no requiriendo más parámetros.

```
Get-ADUser -Filter {(Enabled -eq $False)} |  
Select-Object Name,DistinguishedName,UserPrincipalName,Enabled,SID,LastLogon
```



```
PS C:\Users\Administrador> Get-ADUser -Filter {(Enabled -eq $False)} | Select-Object Name,DistinguishedName,UserPrincipalName,Enabled,SID,LastLogon

Name           : Invitado
DistinguishedName : CN=Invitado,CN=Users,DC=santiago,DC=local
UserPrincipalName :
Enabled        : False
SID            : S-1-5-21-2858253806-180064856-358215610-501
LastLogon      :

Name           : krbtgt
DistinguishedName : CN=krbtgt,CN=Users,DC=santiago,DC=local
UserPrincipalName :
Enabled        : False
SID            : S-1-5-21-2858253806-180064856-358215610-502
LastLogon      :

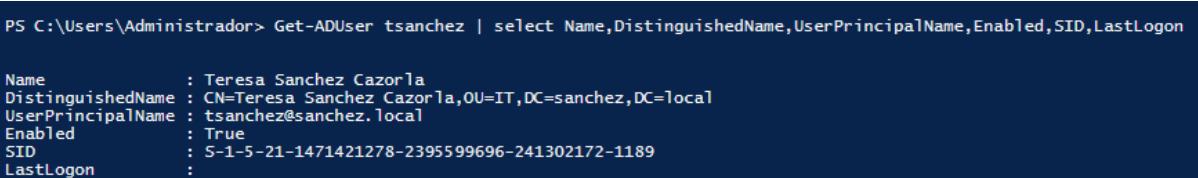
Name           : msagasta
DistinguishedName : CN=msagasta,CN=Users,DC=santiago,DC=local
UserPrincipalName :
Enabled        : False
SID            : S-1-5-21-2858253806-180064856-358215610-1113
LastLogon      :

Name           : vvives
DistinguishedName : CN=vvives,CN=Users,DC=santiago,DC=local
UserPrincipalName :
Enabled        : False
SID            : S-1-5-21-2858253806-180064856-358215610-1114
LastLogon      :
```

16- Listar Último acceso de las cuentas

Podemos mostrar los últimos accesos de los usuarios mediante el siguiente comando.

```
Get-ADUser <nombre_de_usuario> | select  
Name,DistinguishedName,UserPrincipalName,Enabled,SID,LastLogon
```



```
PS C:\Users\Administrador> Get-ADUser tsanchez | select Name,DistinguishedName,UserPrincipalName,Enabled,SID,LastLogon

Name           : Teresa Sanchez Cazorla
DistinguishedName : CN=Teresa Sanchez Cazorla,OU=IT,DC=sanchez,DC=local
UserPrincipalName : tsanchez@sanchez.local
Enabled        : True
SID            : S-1-5-21-1471421278-2395599696-241302172-1189
LastLogon      :
```

Donde [Nombre de usuario] es el usuario sobre el que se quiere consultar.

***no hay captura, al no haber empleado los usuarios de mi active directory.**

17- Listado de usuario bloqueadas

Otro punto para revisar son las cuentas bloqueadas por haber superado un número concreto de intentos de acceso fallidos. Este puede ser un indicativo de que una cuenta tiene usuarios malintencionados que intentan acceder probando contraseñas, aunque posiblemente no sea más que debido a usuarios que cometen varios errores al acceder o no recuerdan su contraseña.

El comando a ejecutar es bastante sencillo, y nuevamente, se basa en

"Search-ADAccount"

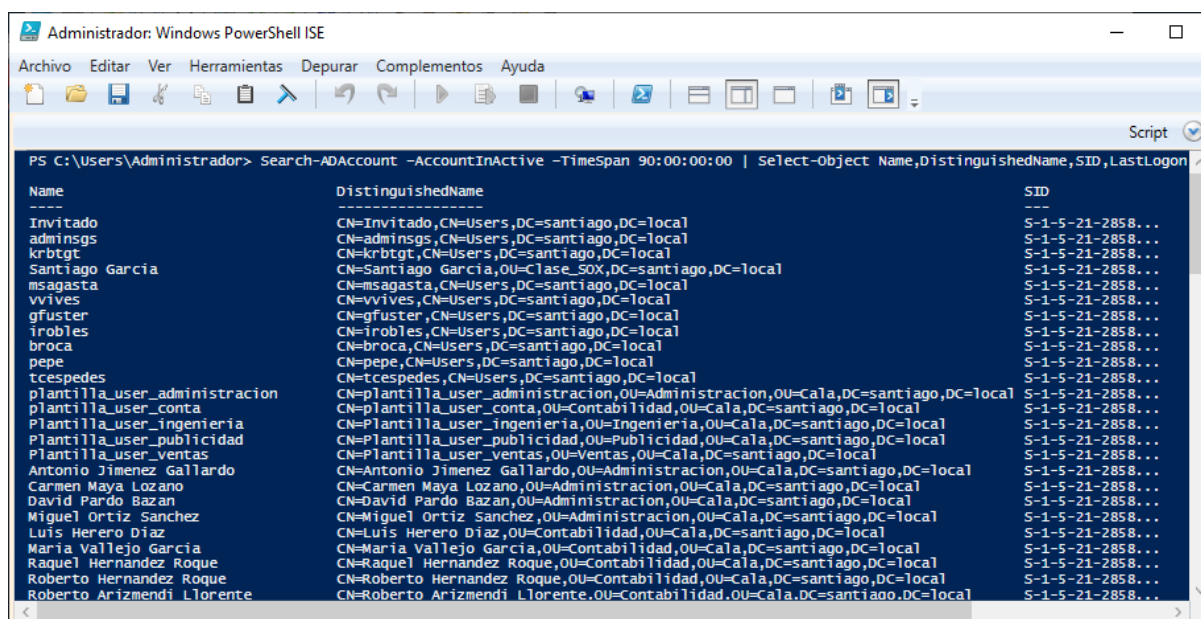
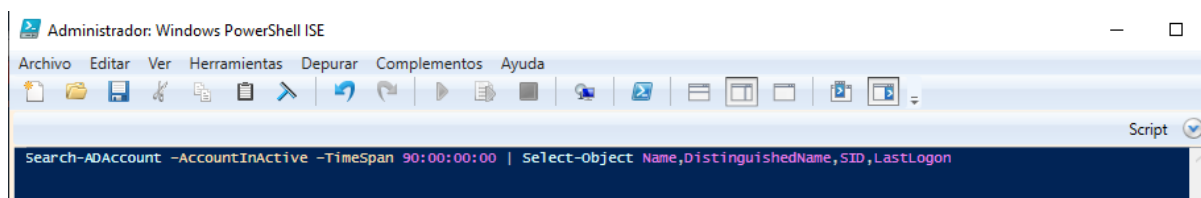
**Search-ADAccount -LockedOut | Select-Object
Name,DistinguishedName,UserPrincipalName,Enabled,SID,LastLogon**

***no hay captura, al no haber empleado los usuarios de mi active directory.**

18- Listado de usuarios inactivos

Dada la reciente creación de nuestro active directory, el tiempo de inactividad será de 90 días. La búsqueda se realiza en base al comando "search-ADAccount"

**Search-ADAccount -AccountInactive -TimeSpan 90:00:00:00 | Select-Object
Name,DistinguishedName,SID,LastLogon**

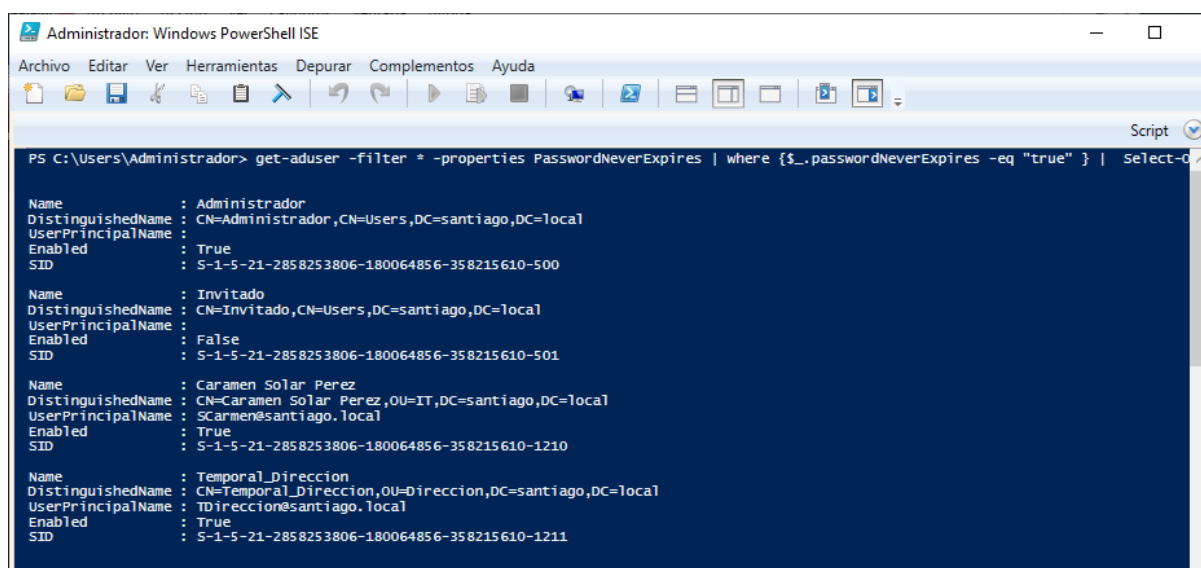


19- Listado de usuarios con password que nunca expira

Las únicas cuentas que deberían permanecer con una contraseña permanente son las asociadas a servicios, ya que su caducidad podría detener los servicios con los que se ejecutan dichos servicios.

Para localizar al resto de los usuarios empleamos el comando “get-aduser” con una serie de parámetros personalizados.

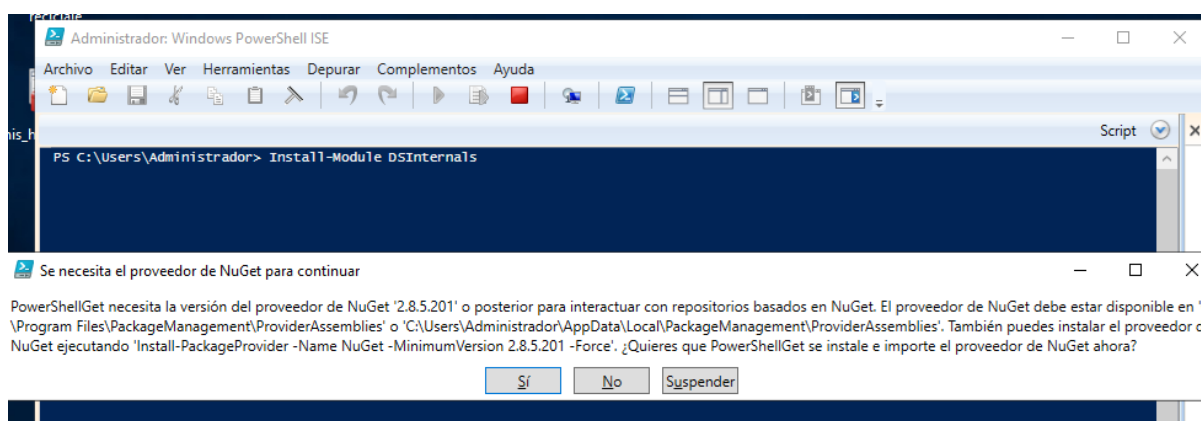
```
get-aduser -filter * -properties PasswordNeverExpires | where  
{$_passwordNeverExpires -eq "true" } | Select-Object  
Name,DistinguishedName,UserPrincipalName,Enabled,SID
```



20- Listado de usuarios con contraseñas débiles

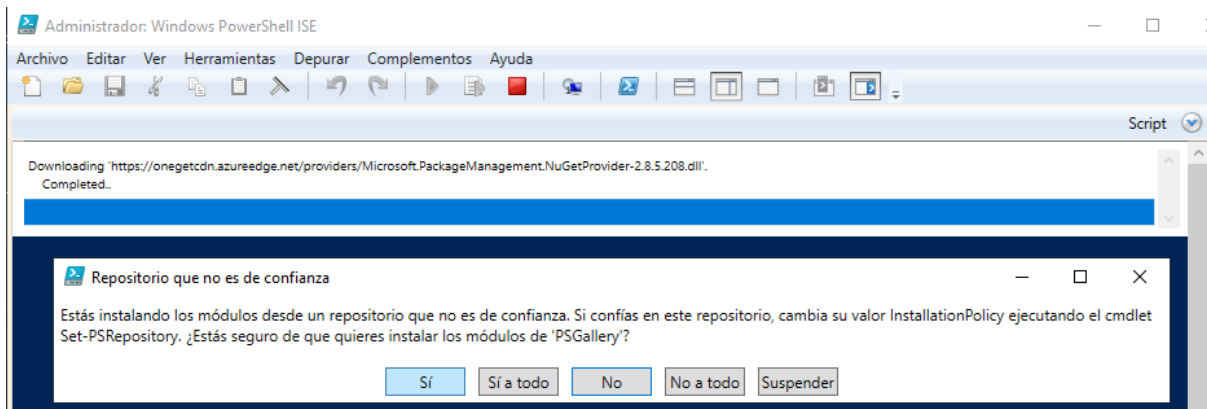
Para realizar esta búsqueda se debe importar el módulo “DSInternals” ejecutando el siguiente comando.

Install-Module DSInternals

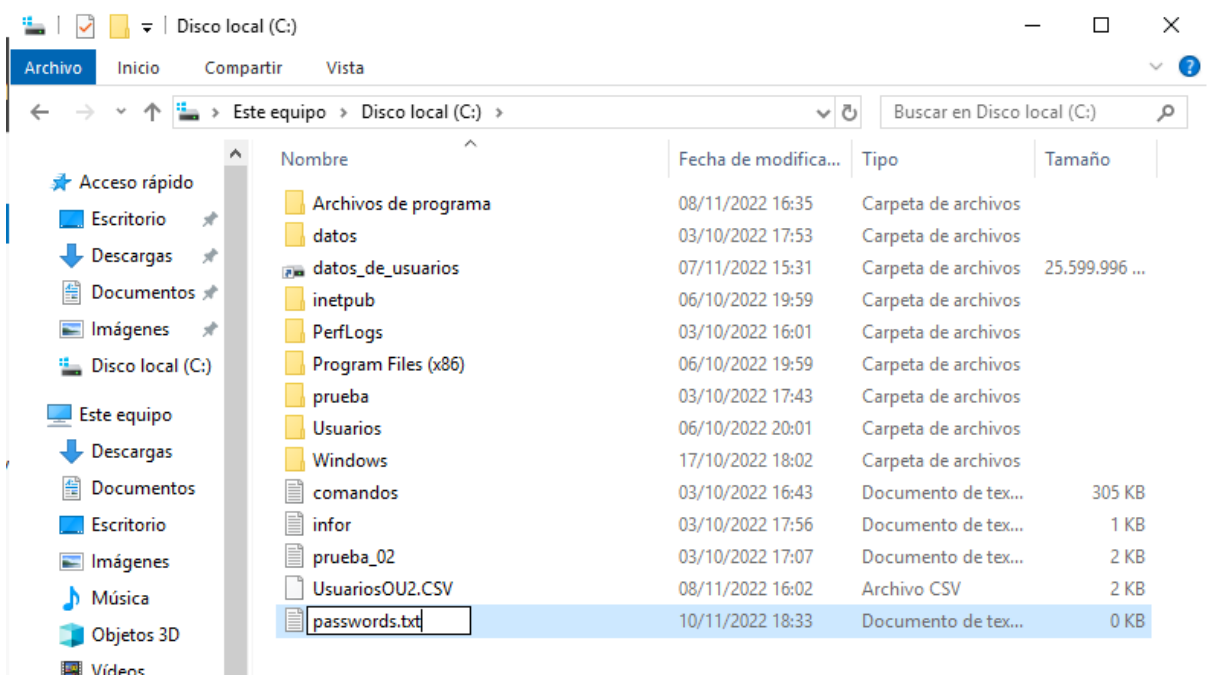
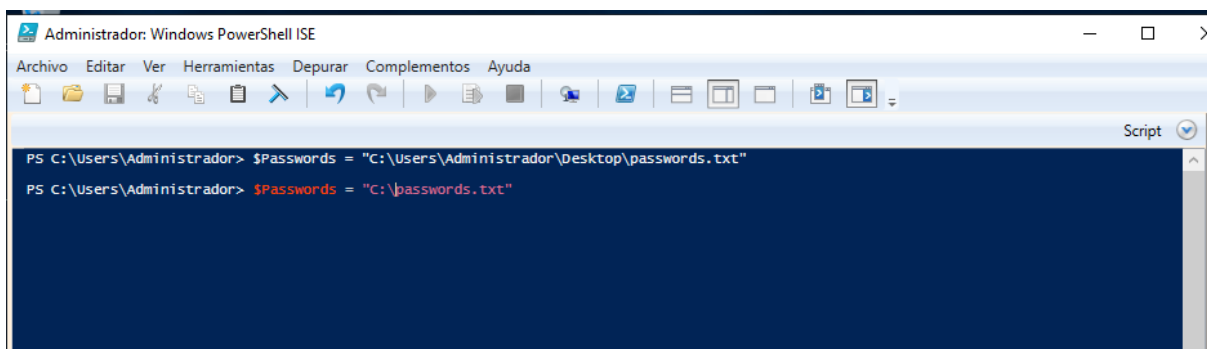


20- Listado de usuarios con contraseñas débiles

[Continuación]



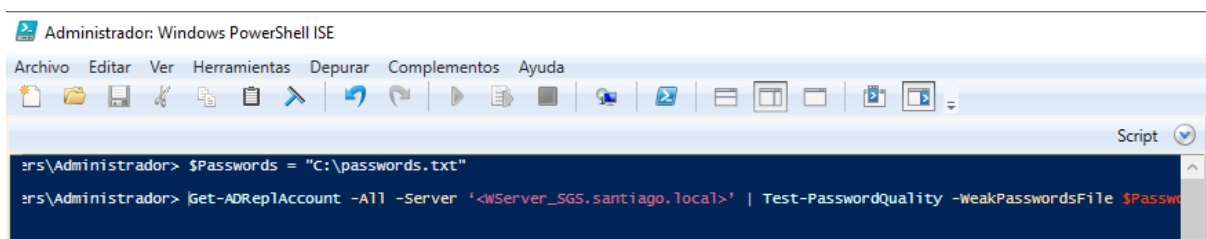
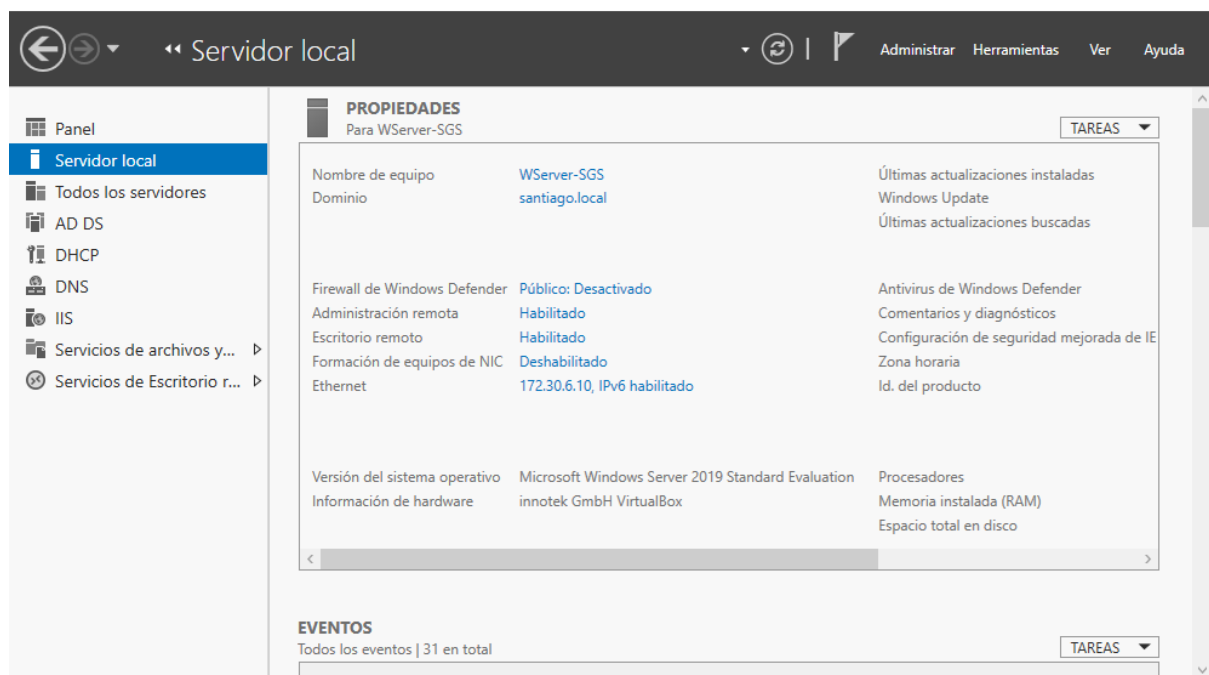
Tras la instalación del módulo, procedemos a crear un .TXT donde estarán almacenados los datos del listado.



20- Listado de usuarios con contraseñas débiles

[Continuación]

Tras la creación del .txt, procedemos a ejecutar el comando que empleando el módulo recién instalado analizará las contraseñas y nos devolverá un informe sobre su nivel de seguridad.

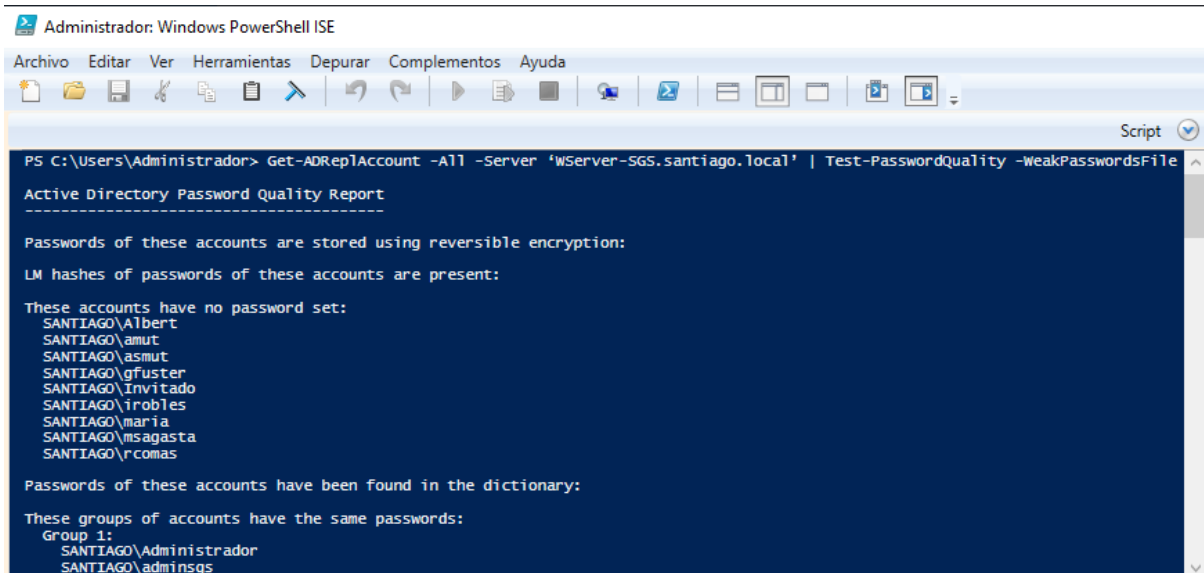


```
Get-ADReplAccount -All -Server '<WServer_SGS.santiago.local>' |  
Test-PasswordQuality -WeakPasswordsFile $Passwords  
-IncludeDisabledAccounts
```

20- Listado de usuarios con contraseñas débiles

[Continuación]

Tras introducir el comando podremos visualizar el primer análisis sobre el estado de nuestras contraseñas, viendo las repetidas y la aparente seguridad de las mismas.



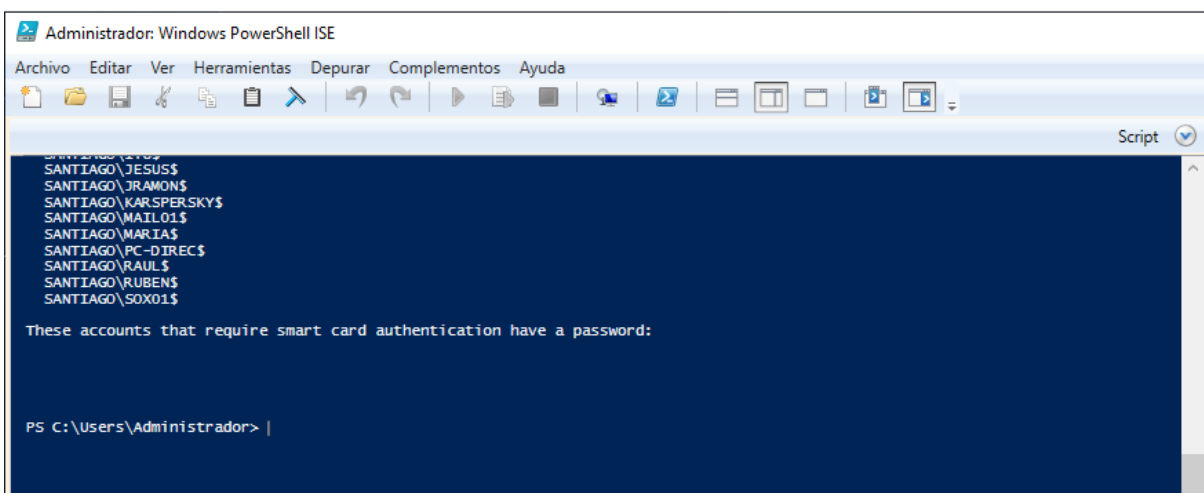
```
PS C:\Users\Administrador> Get-ADRep1Account -All -Server 'WServer-SGS.santiago.local' | Test-PasswordQuality -WeakPasswordsFile

Active Directory Password Quality Report
-----
Passwords of these accounts are stored using reversible encryption:
LM hashes of passwords of these accounts are present:

These accounts have no password set:
SANTIAGO\Albert
SANTIAGO\amut
SANTIAGO\asmut
SANTIAGO\gfuster
SANTIAGO\Invitado
SANTIAGO\irobles
SANTIAGO\maria
SANTIAGO\msagasta
SANTIAGO\rcomas

Passwords of these accounts have been found in the dictionary:

These groups of accounts have the same passwords:
Group 1:
SANTIAGO\Administrador
SANTIAGO\adminsgs
```



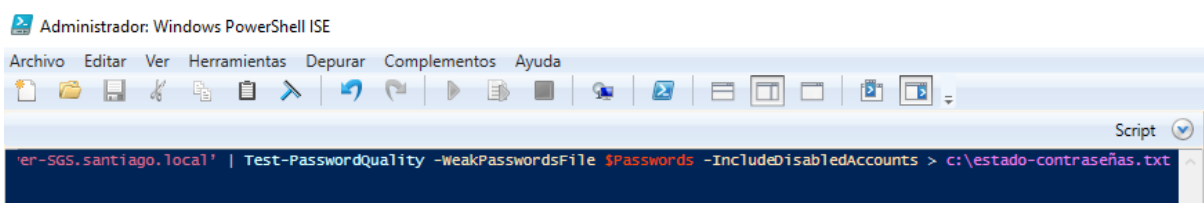
```
SANTIAGO\JESUS$
SANTIAGO\JRAMON$
SANTIAGO\KARSPERSKY$
SANTIAGO\MAIL01$
SANTIAGO\MARIA$
SANTIAGO\PC-DIREC$
SANTIAGO\RAUL$
SANTIAGO\RUBEN$
SANTIAGO\S0X01$

These accounts that require smart card authentication have a password:

PS C:\Users\Administrador> |
```

Una vez ya con estos datos, podemos analizar los datos que hemos extraído en el primer txt y mostrar los resultados del análisis en un segundo txt.

```
Get-ADRep1Account -All -Server 'WServer_SGS.santiago.local' |  
Test-PasswordQuality -WeakPasswordsFile $Passwords  
-IncludeDisabledAccounts > c:\estado-contraseñas.txt
```

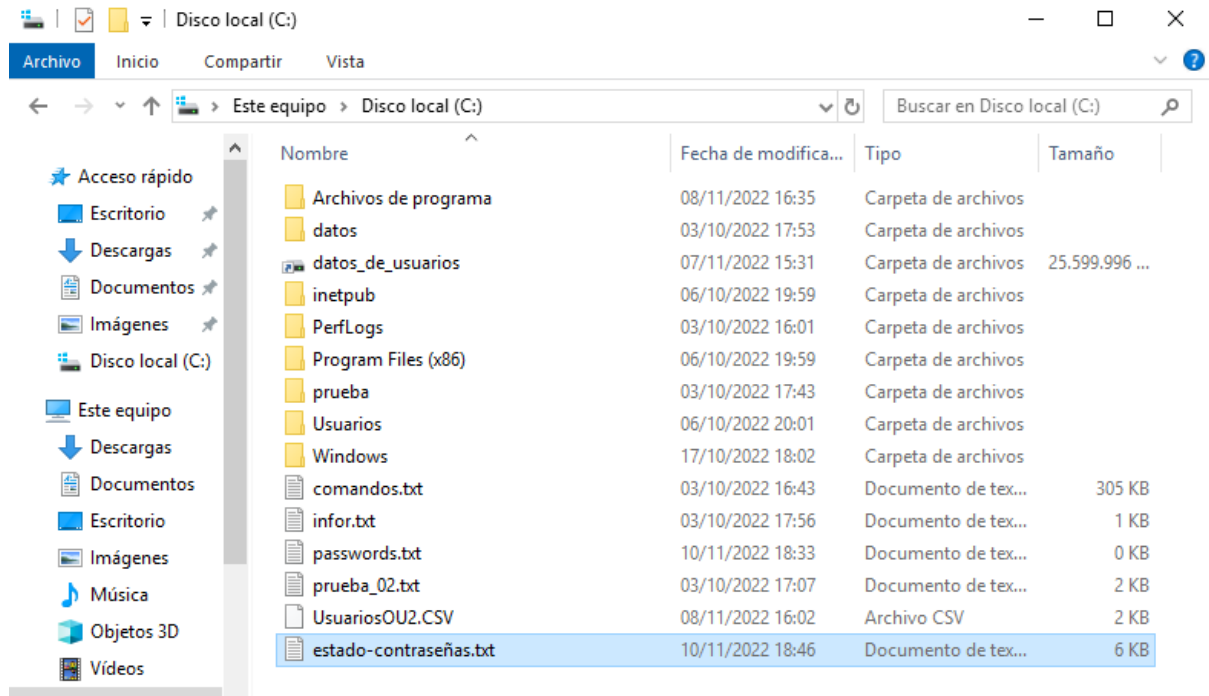


```
er-SGS.santiago.local' | Test-PasswordQuality -WeakPasswordsFile $Passwords -IncludeDisabledAccounts > c:\estado-contraseñas.txt
```

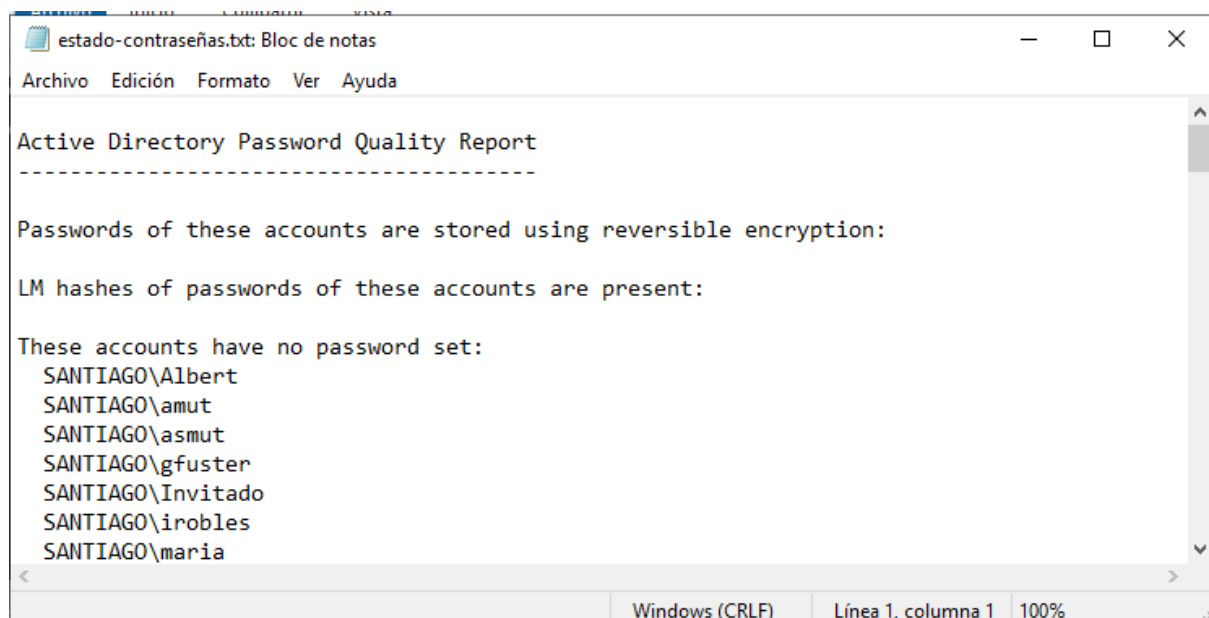
20- Listado de usuarios con contraseñas débiles

[Continuación]

El nuevo archivo con el informe de seguridad ya está disponible en la ubicación que hayamos definido.



Y en su interior, los datos del informe.



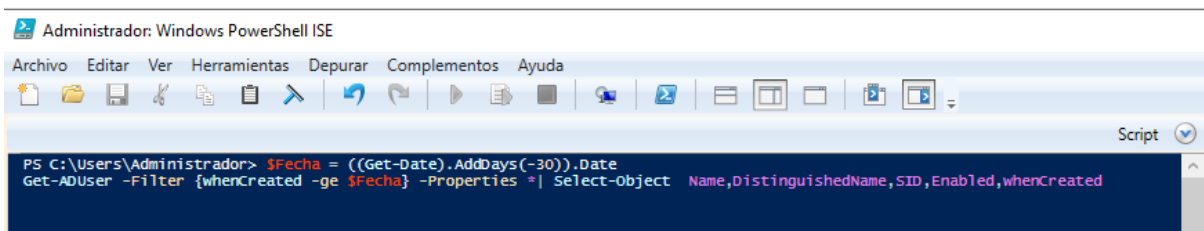
21- Listado de nuevos usuarios

Para hacer esta búsqueda, se puede crear previamente una variable en la que se pase el tiempo de antigüedad de los usuarios. Si, por ejemplo, queremos que se muestren los creados en los últimos 7 días, deberemos usar un valor de "7" para "AddDays", ya que dicha variable, lo que hace es calcular la fecha actual y restarle el valor que hayamos metido en "AddDays".

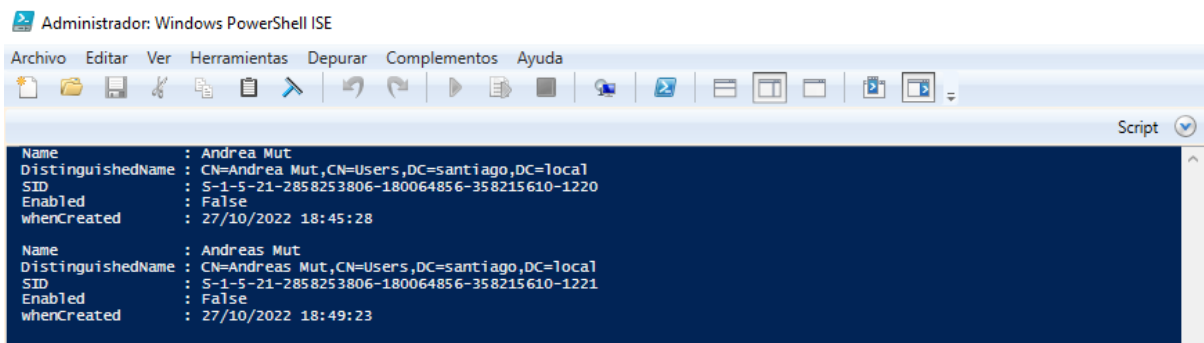
Una vez declarada dicha variable, se usará el comando "Get-ADUser" para hacer una consulta en base al parámetro de creación de la cuenta "whenCreated" en el cual se confronta la fecha frente a la fecha que hemos declarado anteriormente en esa variable.

Si al hacer la comparación, encuentra que el valor obtenido es más grande (-ge) muestra la cuenta, pasando únicamente las propiedades que se han marcado para filtrar.

```
$Fecha = ((Get-Date).AddDays(-30)).Date
Get-ADUser -Filter {whenCreated -ge $Fecha} -Properties * | Select-Object
Name,DistinguishedName,SID,Enabled,whenCreated
```



The screenshot shows the Windows PowerShell ISE interface. The command prompt shows the execution of the script defined above. The command is: `PS C:\Users\Administrador> $Fecha = ((Get-Date).AddDays(-30)).Date; Get-ADUser -Filter {whenCreated -ge $Fecha} -Properties * | Select-Object Name,DistinguishedName,SID,Enabled,whenCreated`



The screenshot shows the output of the command in the PowerShell ISE. The output lists two users: Andrea Mut and Andreas Mut. The properties displayed are Name, DistinguishedName, SID, Enabled, and whenCreated.

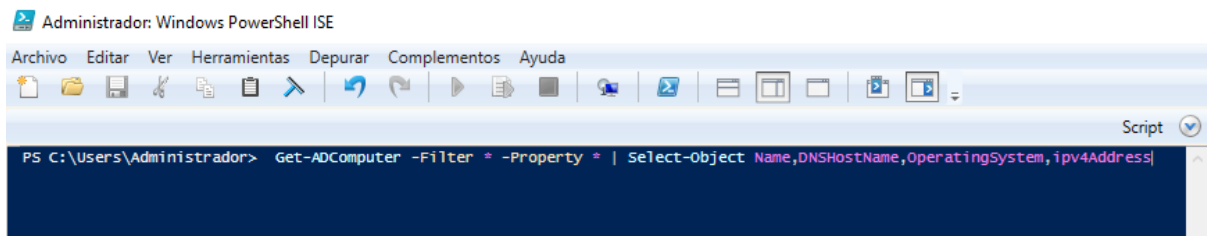
Name	DistinguishedName	SID	Enabled	whenCreated
Andrea Mut	CN=Andrea Mut,CN=Users,DC=santiago,DC=local	S-1-5-21-2858253806-180064856-358215610-1220	False	27/10/2022 18:45:28
Andreas Mut	CN=Andreas Mut,CN=Users,DC=santiago,DC=local	S-1-5-21-2858253806-180064856-358215610-1221	False	27/10/2022 18:49:23

22- Listado de equipos inactivos del dominio

La consulta es bastante simple y básicamente consiste en lanzar el comando **"Get-ADComputer"** que lista los equipos y servidores (todos los elementos dados de alta como "computer") filtrando sus propiedades y sacando los elementos que nos parezcan relevantes.

El comando quedaría conformado de la siguiente manera:

```
Get-ADComputer -Filter * -Property * | Select-Object  
Name,DNSHostName,OperatingSystem,ipv4Address
```

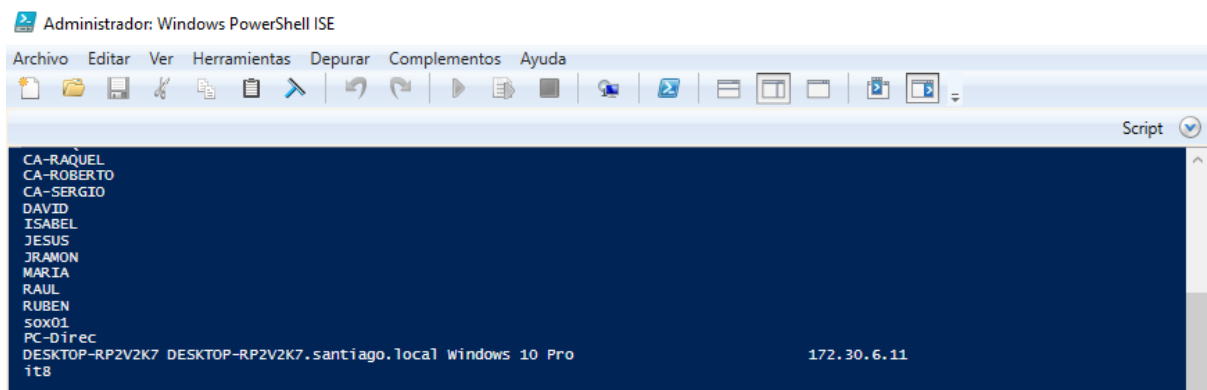


Administrador: Windows PowerShell ISE

Archivo Editar Ver Herramientas Depurar Complementos Ayuda

Script

```
PS C:\Users\Administrador> Get-ADComputer -Filter * -Property * | Select-Object Name,DNSHostName,OperatingSystem,ipv4Address
```



Administrador: Windows PowerShell ISE

Archivo Editar Ver Herramientas Depurar Complementos Ayuda

Script

```
CA-RAQUEL  
CA-ROBERTO  
CA-SERGIO  
DAVID  
ISABEL  
JESUS  
JRAMON  
MARIA  
RAUL  
RUBEN  
sox01  
PC-Direc  
DESKTOP-RP2V2K7 DESKTOP-RP2V2K7.santiago.local Windows 10 Pro 172.30.6.11  
it8
```

23- Listado de equipos deshabilitados

Para localizarlos basta con hacer una consulta con "Get-ADComputer" filtrando aquellos que tengan la propiedad "Enabled" como falsa.

Get-ADComputer -Filter {(Enabled -eq \$False)} | Select-Object Name

